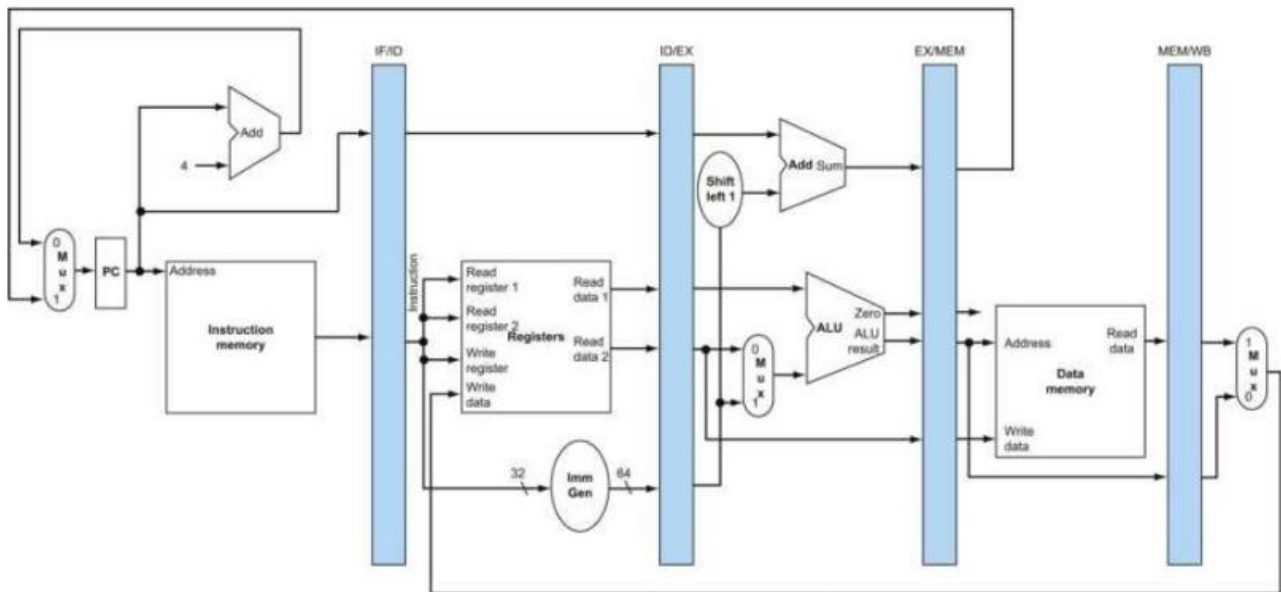


5-Stage Pipelined RISC-V Processor



Pipelined Processor for R-type instruction (add)

```

module pipelined_processor(
    input clk,
    input rst
);
    wire [31:0] Nxt_PC;
    reg [31:0] PC_register;
    // Memory array: 256 bytes total
    reg [7:0] Inst_memory [0:255];
    // Wire to hold the raw fetched instruction from memory
    wire [31:0] current_instruction;
    // IF/ID Pipeline Registers
    reg [31:0] IFID_instruction;
    reg [31:0] IFID_PC;
    // ID/EX Pipeline Registers
    reg [31:0] IDEX_reg_data1;
    reg [31:0] IDEX_reg_data2;
    reg [4:0] IDEX_rd;
    reg [2:0] IDEX_funct3;
    reg [6:0] IDEX_funct7;
    // EX/MEM Pipeline Registers
    reg [31:0] EXMEM_alu_result;
    reg [4:0] EXMEM_rd;
    // MEM/WB Pipeline Registers
    reg [31:0] MEMWB_alu_result;
    reg [4:0] MEMWB_rd;
    // Operand
    wire [6:0] opcode = IFID_instruction[6:0];
    wire [4:0] rd = IFID_instruction[11:7];
    wire [2:0] funct3 = IFID_instruction[14:12];
    wire [4:0] rs1 = IFID_instruction[19:15];
    wire [4:0] rs2 = IFID_instruction[24:20];
    wire [6:0] funct7 = IFID_instruction[31:25];
    // Register file
    reg [31:0] Register_File [0:31];
    wire [31:0] reg_data1;
    
```

```

wire [31:0] reg_data2;

// If rs1 is 0, output 0; otherwise, read from the register file array
assign reg_data1 = (rs1 == 5'b0) ? 32'h0 : Register_File[rs1];
assign reg_data2 = (rs2 == 5'b0) ? 32'h0 : Register_File[rs2];

// ALU Result
reg [31:0] alu_result;
// Combinational PC Adder (PC + 4)
assign Nxt_PC = PC_register + 4;

// Sequential PC Register Update
always @(posedge clk) begin
    if (rst)
        PC_register <= 32'h0;    // Reset PC to address 0
    else
        PC_register <= Nxt_PC;    // Move to next sequential address
    end
// Combinational Instruction Fetch from Byte-Addressable Memory
assign current_instruction = {Inst_memory[PC_register + 3], Inst_memory[PC_register + 2],
Inst_memory[PC_register + 1], Inst_memory[PC_register] };

always @(posedge clk) begin
    if (rst) begin
        IFID_instruction <= 32'h0; // Flush instruction on reset (NOP)
        IFID_PC <= 32'h0;
    end else begin
        IFID_instruction <= current_instruction; // Pass instruction to next stage
        IFID_PC <= PC_register;    // Pass current instruction's PC
    end
end

always @(posedge clk) begin
    if (rst) begin
        IDEX_reg_data1 <= 32'h0;
        IDEX_reg_data2 <= 32'h0;
        IDEX_rd <= 5'h0;
        IDEX_funct3 <= 3'h0;
        IDEX_funct7 <= 7'h0;
    end else begin
        IDEX_reg_data1 <= reg_data1; // Capture read operand 1
        IDEX_reg_data2 <= reg_data2; // Capture read operand 2
        IDEX_rd <= rd;    // Pass destination register index downstream
        IDEX_funct3 <= funct3; // Pass funct3 for ALU control
        IDEX_funct7 <= funct7; // Pass funct7 for ALU control
    end
end

always @* begin
    alu_result = 32'h0;
    if( IDEX_funct3 == 3'b000 && IDEX_funct7 == 7'b0000000) begin
        alu_result = IDEX_reg_data1 + IDEX_reg_data2;
    end
end

always @(posedge clk) begin
    if (rst) begin
        EXMEM_alu_result <= 32'h0;
        EXMEM_rd <= 5'h0;
    end
end

```

```

end else begin
    EXMEM_alu_result <= alu_result;
    EXMEM_rd      <= IDEX_rd;
end
end
always @(posedge clk) begin
    if(rst) begin
        MEMWB_alu_result <= 32'h0;
        MEMWB_rd <= 5'h0; end
    else begin
        MEMWB_alu_result <= EXMEM_alu_result;
        MEMWB_rd <= EXMEM_rd;
    end
end
always @(posedge clk) begin
    if (!rst && MEMWB_rd != 5'b0) begin
        Register_File[MEMWB_rd] <= MEMWB_alu_result;
    end
end
end        endmodule

```

Testbench:

```

`timescale 1ns / 1ps
module tb_pipelined_processor();
    // Testbench Inputs (Regs)
    reg clk;
    reg rst;
    // Instantiate the Unit Under Test (UUT)
    pipelined_processor uut (
        .clk(clk),
        .rst(rst)
    );
    initial begin
        $dumpfile( "dump.vcd");
        $dumpvars(0, tb_pipelined_processor);
    end
    // 1. Clock Generation (10ns period -> 100MHz clock)
    always begin
        #5 clk = ~clk;
    end
    // 2. Main Simulation Sequence
    initial begin
        // Initialize inputs
        clk = 0;
        rst = 1;
        // Wait 20ns for a stable system startup reset condition
        #20;
        // Pre-populate source registers using hierarchical paths
        // Let's add: x1 = 15 (0x0F) and x2 = 25 (0x19)
        // Expected outcome: x3 = 15 + 25 = 40 (0x28)
        uut.Register_File[1] = 32'd15;
        uut.Register_File[2] = 32'd25;
        // Explicitly load the 32-bit instruction code into Byte-Addressable memory
        // Assembly: add x3, x1, x2 -> Hex: 32'h002081b3
        // Stored in Little-Endian layout starting at Address 0
        uut.Inst_memory[0] = 8'hb3;
        uut.Inst_memory[1] = 8'h81;
        uut.Inst_memory[2] = 8'h20;
    end
endmodule

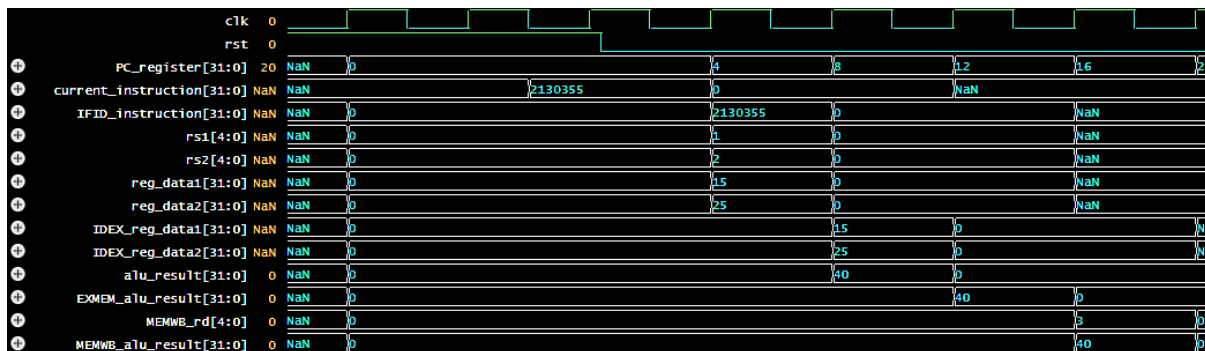
```

```

    uut.Inst_memory[3] = 8'h00;
    // Initialize trailing memory locations with NOP instructions (all zeros)
    // to prevent the processor from fetching garbage data on subsequent cycles.
    uut.Inst_memory[4] = 8'h00; uut.Inst_memory[5] = 8'h00; uut.Inst_memory[6] = 8'h00; uut.Inst_memory[7] =
8'h00;
    uut.Inst_memory[8] = 8'h00; uut.Inst_memory[9] = 8'h00; uut.Inst_memory[10] = 8'h00; uut.Inst_memory[11] =
8'h00;
    // Release reset to start execution
    @(posedge clk);
    #1 rst = 0;
    // Let the simulation run for 8 clock cycles to witness the progression
    // through Fetch, Decode, Execute, Memory, and Write-Back stages.
    repeat (8) @(posedge clk);
    // End simulation safely
    $finish;
end
// 3. Automated Runtime Monitoring Loop
// This block prints out the state of key pipeline parameters on every cycle.
always @(posedge clk) begin
    if (!rst) begin
        $display("--- Time: %0t ns ---", $time);
        $display("Fetch | PC: %d | Raw Inst Captured: %h", uut.PC_register, uut.current_instruction);
        $display("Decode | rs1 Data: %d | rs2 Data: %d | Target rd: x%0d", uut.IDEX_reg_data1, uut.IDEX_reg_data2,
uut.IDEX_rd);
        $display("Exec | Calculated ALU Result: %d", uut.EXMEM_alu_result);
        $display("WriteB | RegFile[x3] Value: %d", uut.Register_File[3]);
        $display("-----\n");
    end
end
endmodule

```

Output:



VCD info: dumpfile dump.vcd opened for output.

--- Time: 35000 ns ---

```

Fetch | PC:          0 | Raw Inst Captured: 002081b3
Decode | rs1 Data:      0 | rs2 Data:      0 | Target rd: x0
Exec   | Calculated ALU Result:      0
WriteB | RegFile[x3] value:      x

```

--- Time: 45000 ns ---

```

Fetch | PC:          4 | Raw Inst Captured: 00000000
Decode | rs1 Data:      0 | rs2 Data:      0 | Target rd: x0
Exec   | Calculated ALU Result:      0
WriteB | RegFile[x3] value:      x

```

--- Time: 55000 ns ---

```

Fetch | PC:          8 | Raw Inst Captured: 00000000
Decode | rs1 Data:     15 | rs2 Data:     25 | Target rd: x3
Exec   | Calculated ALU Result:      0
WriteB | RegFile[x3] value:      x

```

```

--- Time: 65000 ns ---
Fetch | PC:          12 | Raw Inst Captured: xxxxxxxx
Decode | rs1 Data:      0 | rs2 Data:          0 | Target rd: x0
Exec   | Calculated ALU Result: 40
WriteB | RegFile[x3] Value:      x
-----
--- Time: 75000 ns ---
Fetch | PC:          16 | Raw Inst Captured: xxxxxxxx
Decode | rs1 Data:      0 | rs2 Data:          0 | Target rd: x0
Exec   | Calculated ALU Result:  0
WriteB | RegFile[x3] Value:      x
-----
--- Time: 85000 ns ---
Fetch | PC:          20 | Raw Inst Captured: xxxxxxxx
Decode | rs1 Data:      x | rs2 Data:          x | Target rd: xx
Exec   | Calculated ALU Result:  0
WriteB | RegFile[x3] Value:     40
-----
--- Time: 95000 ns ---
Fetch | PC:          24 | Raw Inst Captured: xxxxxxxx
Decode | rs1 Data:      x | rs2 Data:          x | Target rd: xx
Exec   | Calculated ALU Result:  0
WriteB | RegFile[x3] Value:     40
-----
--- Time: 105000 ns ---
Fetch | PC:          28 | Raw Inst Captured: xxxxxxxx
Decode | rs1 Data:      x | rs2 Data:          x | Target rd: xx
Exec   | Calculated ALU Result:  0
WriteB | RegFile[x3] Value:     40
-----

```

Pipelined Processor for I-type instruction (addi)

```

module pipelined_processor(
    input clk,
    input rst
);
    wire [31:0] Nxt_PC;
    reg [31:0] PC_register;
    // Memory array: 256 bytes total
    reg [7:0] Inst_memory [0:255];
    // Wire to hold the raw fetched instruction from memory
    wire [31:0] current_instruction;
    // IF/ID Pipeline Registers
    reg [31:0] IFID_instruction;
    reg [31:0] IFID_PC;
    // ID/EX Pipeline Registers
    reg [31:0] IDEX_reg_data1;
    reg [31:0] IDEX_reg_data2;
    reg [4:0] IDEX_rd;
    reg [2:0] IDEX_func3;
    reg [6:0] IDEX_func7;
    reg [6:0] IDEX_opcode;
    // EX/MEM Pipeline Registers
    reg [31:0] EXMEM_alu_result;
    reg [4:0] EXMEM_rd;
    // MEM/WB Pipeline Registers
    reg [31:0] MEMWB_alu_result;
    reg [4:0] MEMWB_rd;
    // Operand for R-type
    wire [6:0] opcode = IFID_instruction[6:0];
    wire [4:0] rd = IFID_instruction[11:7];
    wire [2:0] func3 = IFID_instruction[14:12];
    wire [4:0] rs1 = IFID_instruction[19:15];

```

```

        wire [4:0] rs2 = IFID_instruction[24:20];
        wire [6:0] funct7 = IFID_instruction[31:25];
// Operand for I-type
wire [31:0] imm_ext = { {20{IFID_instruction[31]}}, IFID_instruction[31:20] };
// Register file
reg [31:0] Register_File [0:31];
wire [31:0] reg_data1;
        wire [31:0] reg_data2;
        // If rs1 is 0, output 0; otherwise, read from the register file array
        assign reg_data1 = (rs1 == 5'b0) ? 32'h0 : Register_File[rs1];
        assign reg_data2 = (rs2 == 5'b0) ? 32'h0 : Register_File[rs2];
// ALU Result
reg [31:0] alu_result;
reg [31:0] IDEX_imm_ext;
// Combinational PC Adder (PC + 4)
assign Nxt_PC = PC_register + 4;
// Sequential PC Register Update
always @(posedge clk) begin
    if (rst)
        PC_register <= 32'h0;    // Reset PC to address 0
    else
        PC_register <= Nxt_PC;    // Move to next sequential address
end
// Combinational Instruction Fetch from Byte-Addressable Memory
assign current_instruction = {Inst_memory[PC_register + 3], Inst_memory[PC_register + 2],
Inst_memory[PC_register + 1], Inst_memory[PC_register] };
always @(posedge clk) begin
    if (rst) begin
        IFID_instruction <= 32'h0; // Flush instruction on reset (NOP)
        IFID_PC <= 32'h0;
    end else begin
        IFID_instruction <= current_instruction; // Pass instruction to next stage
        IFID_PC <= PC_register;    // Pass current instruction's PC
    end
end
always @(posedge clk) begin
    if (rst) begin
        IDEX_reg_data1 <= 32'h0;
        IDEX_reg_data2 <= 32'h0;
        IDEX_imm_ext <= 32'h0;
        IDEX_rd <= 5'h0;
        IDEX_funct3 <= 3'h0;
        IDEX_funct7 <= 7'h0;
        IDEX_opcode <= 7'h0;
    end else begin
        IDEX_reg_data1 <= reg_data1;
        IDEX_reg_data2 <= reg_data2;
        IDEX_imm_ext <= imm_ext;
        IDEX_rd <= rd;
        IDEX_funct3 <= funct3;
        IDEX_funct7 <= funct7;
        IDEX_opcode <= opcode; // Passing opcode downstream safely!
    end
end
wire [31:0] alu_operand2;
assign alu_operand2 = (IDEX_opcode == 7'b0010011) ? IDEX_imm_ext : IDEX_reg_data2;

```

```

always @* begin
    alu_result = 32'h0;
    if (IDEX_opcode == 7'b0110011 && IDEX_func3 == 3'b000 && IDEX_func7 == 7'b0000000) begin
        alu_result = IDEX_reg_data1 + alu_operand2;
    end
    else if (IDEX_opcode == 7'b0010011 && IDEX_func3 == 3'b000) begin
        alu_result = IDEX_reg_data1 + alu_operand2;
    end
end
end

```

```

always @(posedge clk) begin
    if (rst) begin
        EXMEM_alu_result <= 32'h0;
        EXMEM_rd <= 5'h0;
    end else begin
        EXMEM_alu_result <= alu_result;
        EXMEM_rd <= IDEX_rd;
    end
end
end

```

```

always @(posedge clk) begin
    if(rst) begin
        MEMWB_alu_result <= 32'h0;
        MEMWB_rd <= 5'h0; end
    else begin
        MEMWB_alu_result <= EXMEM_alu_result;
        MEMWB_rd <= EXMEM_rd;
    end
end
end

```

```

always @(posedge clk) begin
    if (!rst && MEMWB_rd != 5'b0) begin
        Register_File[MEMWB_rd] <= MEMWB_alu_result;
    end
end
end endmodule

```

Testbench:

```

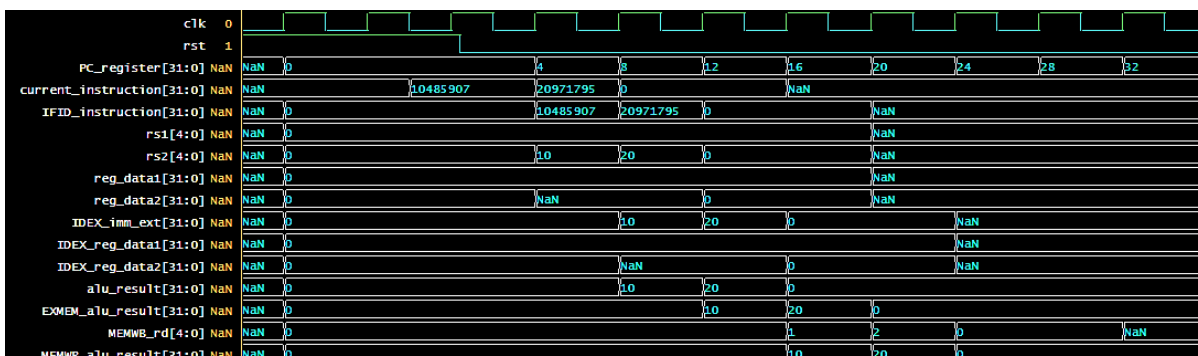
`timescale 1ns / 1ps
module tb_pipelined_processor();
    // Testbench Inputs (Regs)
    reg clk;
    reg rst;
    // Instantiate the Unit Under Test (UUT)
    pipelined_processor uut (
        .clk(clk),
        .rst(rst)
    );
    // VCD Dump configuration for EPWave visualization
    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_pipelined_processor);
    end
    // 1. Clock Generation (10ns period -> 100MHz clock)
    always begin
        #5 clk = ~clk;
    end
    // 2. Main Simulation Sequence

```

```

initial begin
    // Initialize inputs
    clk = 0;
    rst = 1;
    // Wait 20ns for a stable system startup reset condition
    #20;
    // --- INSTRUCTION 1: addi x1, x0, 10 ---
    // Machine Code Hex: 32'h00A00093
    // Stored in Little-Endian starting at Address 0
    uut.Inst_memory[0] = 8'h93;
    uut.Inst_memory[1] = 8'h00;
    uut.Inst_memory[2] = 8'hA0;
    uut.Inst_memory[3] = 8'h00;
    // --- INSTRUCTION 2: addi x2, x0, 20 ---
    // Machine Code Hex: 32'h01400113
    // Stored in Little-Endian starting at Address 4
    uut.Inst_memory[4] = 8'h13;
    uut.Inst_memory[5] = 8'h01;
    uut.Inst_memory[6] = 8'h40;
    uut.Inst_memory[7] = 8'h01;
    // Initialize trailing memory locations with NOP instructions (all zeros)
    uut.Inst_memory[8] = 8'h00; uut.Inst_memory[9] = 8'h00; uut.Inst_memory[10] = 8'h00; uut.Inst_memory[11]
= 8'h00;
    uut.Inst_memory[12] = 8'h00; uut.Inst_memory[13] = 8'h00; uut.Inst_memory[14] = 8'h00;
    uut.Inst_memory[15] = 8'h00;
    // Release reset to start execution
    @(posedge clk);
    #1 rst = 0;
    // Running for 12 clock cycles to let both addi instructions complete their paths
    repeat (12) @(posedge clk);
    // End simulation safely
    $finish;
end
// 3. Automated Runtime Monitoring Loop
always @(posedge clk) begin
    if (!rst) begin
        $display("--- Time: %0t ns ---", $time);
        $display("Fetch | PC: %d | Raw Inst Captured: %h", uut.PC_register, uut.current_instruction);
        $display("Decode | Opcode: %b | rs1 Data: %d | Immediate: %d | Target rd: x%0d",
            uut.opcode, uut.IDEX_reg_data1, uut.IDEX_imm_ext, uut.IDEX_rd);
        $display("Exec | Calculated ALU Result: %d", uut.EXMEM_alu_result);
        $display("WriteB | RegFile[x1] = %d | RegFile[x2] = %d", uut.Register_File[1], uut.Register_File[2]);
        $display("-----\n");
    end
end
endmodule

```



```

VCD info: dumpfile dump.vcd opened for output.
--- Time: 35000 ns ---
Fetch | PC:          0 | Raw Inst Captured: 00a00093
Decode | Opcode: 0000000 | rs1 Data:          0 | Immediate:          0 | Target rd: x0
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1] =          x | RegFile[x2] =          x
-----
--- Time: 45000 ns ---
Fetch | PC:          4 | Raw Inst Captured: 01400113
Decode | Opcode: 0010011 | rs1 Data:          0 | Immediate:          0 | Target rd: x0
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1] =          x | RegFile[x2] =          x
-----
--- Time: 55000 ns ---
Fetch | PC:          8 | Raw Inst Captured: 00000000
Decode | Opcode: 0010011 | rs1 Data:          0 | Immediate:          0 | Target rd: x1
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1] =          x | RegFile[x2] =          x
-----
--- Time: 65000 ns ---
Fetch | PC:         12 | Raw Inst Captured: 00000000
Decode | Opcode: 0000000 | rs1 Data:          0 | Immediate:          0 | Target rd: x2
Exec   | Calculated ALU Result:         10
WriteB | RegFile[x1] =          x | RegFile[x2] =          x
-----
--- Time: 75000 ns ---
Fetch | PC:         16 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: 0000000 | rs1 Data:          0 | Immediate:          0 | Target rd: x0
Exec   | Calculated ALU Result:         20
WriteB | RegFile[x1] =          x | RegFile[x2] =          x
-----
--- Time: 85000 ns ---
Fetch | PC:         20 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data:          0 | Immediate:          0 | Target rd: x0
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1] =         10 | RegFile[x2] =          x
-----
--- Time: 95000 ns ---
Fetch | PC:         24 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data:          x | Immediate:          0 | Target rd: xx
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1] =         10 | RegFile[x2] =         20
-----
--- Time: 105000 ns ---
Fetch | PC:         28 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data:          x | Immediate:          0 | Target rd: xx
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1] =         10 | RegFile[x2] =         20
-----
--- Time: 115000 ns ---
Fetch | PC:         32 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data:          x | Immediate:          0 | Target rd: xx
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1] =         10 | RegFile[x2] =         20
-----
--- Time: 125000 ns ---
Fetch | PC:         36 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data:          x | Immediate:          0 | Target rd: xx
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1] =         10 | RegFile[x2] =         20
-----
--- Time: 135000 ns ---
Fetch | PC:         40 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data:          x | Immediate:          0 | Target rd: xx
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1] =         10 | RegFile[x2] =         20
-----
--- Time: 145000 ns ---
Fetch | PC:         44 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data:          x | Immediate:          0 | Target rd: xx
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1] =         10 | RegFile[x2] =         20
-----

```

Creating a Data Hazard - RAW (Read After Write)

```
`timescale 1ns / 1ps
```

```

module tb_pipelined_processor();
    // Testbench Inputs (Regs)
    reg clk;

```

```

reg rst;
// Instantiate the Unit Under Test (UUT)
pipelined_processor uut (
    .clk(clk),
    .rst(rst)
);
// VCD Dump configuration for EPWave visualization
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_pipelined_processor);
end
// 1. Clock Generation (10ns period -> 100MHz clock)
always begin
    #5 clk = ~clk;
end
// 2. Main Simulation Sequence
initial begin
    // Initialize inputs
    clk = 0;
    rst = 1;
    // Wait 20ns for a stable system startup reset condition
    #20;
    // --- INSTRUCTION 1: addi x1, x0, 10 ---
    // Machine Code Hex: 32'h00A00093
    uut.Inst_memory[0] = 8'h93;
    uut.Inst_memory[1] = 8'h00;
    uut.Inst_memory[2] = 8'hA0;
    uut.Inst_memory[3] = 8'h00;
    // --- INSTRUCTION 2: addi x2, x0, 20 ---
    // Machine Code Hex: 32'h01400113
    uut.Inst_memory[4] = 8'h13;
    uut.Inst_memory[5] = 8'h01;
    uut.Inst_memory[6] = 8'h40;
    uut.Inst_memory[7] = 8'h01;
    // --- INSTRUCTION 3: add x3, x1, x2 ---
    // Machine Code Hex: 32'h002081b3
    // Stored in Little-Endian starting at Address 8
    uut.Inst_memory[8] = 8'hb3;
    uut.Inst_memory[9] = 8'h81;
    uut.Inst_memory[10] = 8'h20;
    uut.Inst_memory[11] = 8'h00;
    // Initialize trailing memory locations with NOP instructions (all zeros)
    uut.Inst_memory[12] = 8'h00; uut.Inst_memory[13] = 8'h00; uut.Inst_memory[14] = 8'h00;
    uut.Inst_memory[15] = 8'h00;
    uut.Inst_memory[16] = 8'h00; uut.Inst_memory[17] = 8'h00; uut.Inst_memory[18] = 8'h00;
    uut.Inst_memory[19] = 8'h00;
    // Release reset to start execution
    @(posedge clk);
    #1 rst = 0;
    // Running for 14 clock cycles to observe the hazard manifestation
    repeat (14) @(posedge clk);
    // End simulation safely
    $finish;
end
// 3. Automated Runtime Monitoring Loop
always @(posedge clk) begin
    if (!rst) begin

```

```

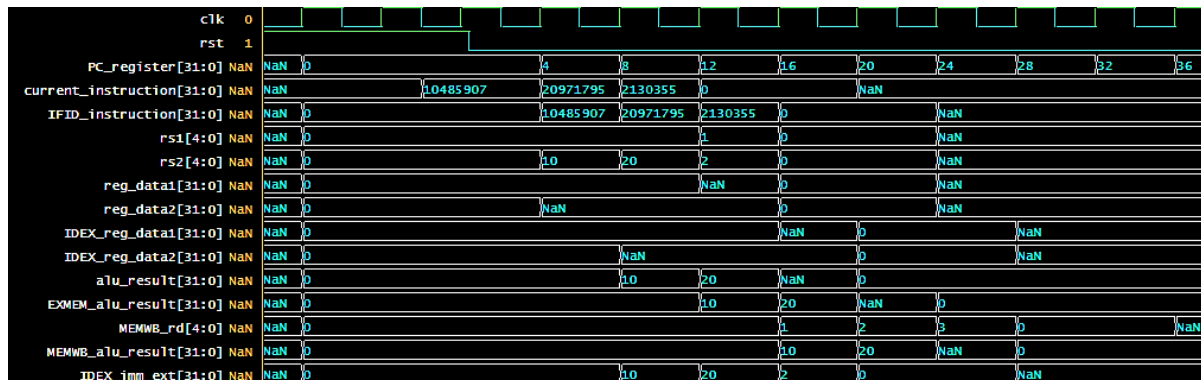
$display("--- Time: %0t ns ---", $time);
$display("Fetch | PC: %d | Raw Inst Captured: %h", uut.PC_register, uut.current_instruction);
$display("Decode | Opcode: %b | rs1 Data: %d | rs2 Data: %d | Target rd: x%0d",
    uut.opcode, uut.reg_data1, uut.reg_data2, uut.rd);
$display("Exec | Calculated ALU Result: %d", uut.EXMEM_alu_result);
$display("WriteB | RegFile[x1]=%d | RegFile[x2]=%d | RegFile[x3]=%d",
    uut.Register_File[1], uut.Register_File[2], uut.Register_File[3]);
$display("-----\n");

```

```
end
```

```
end
```

```
endmodule
```



```

--- Time: 35000 ns ---
Fetch | PC: 0 | Raw Inst Captured: 00a00093
Decode | Opcode: 0000000 | rs1 Data: 0 | rs2 Data: 0 | Target rd: x0
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x
-----
--- Time: 45000 ns ---
Fetch | PC: 4 | Raw Inst Captured: 01400113
Decode | Opcode: 0010011 | rs1 Data: 0 | rs2 Data: x | Target rd: x1
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x
-----
--- Time: 55000 ns ---
Fetch | PC: 8 | Raw Inst Captured: 002081b3
Decode | Opcode: 0010011 | rs1 Data: 0 | rs2 Data: x | Target rd: x2
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x
-----
--- Time: 65000 ns ---
Fetch | PC: 12 | Raw Inst Captured: 00000000
Decode | Opcode: 0110011 | rs1 Data: x | rs2 Data: x | Target rd: x3
Exec | Calculated ALU Result: 10
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x
-----
--- Time: 75000 ns ---
Fetch | PC: 16 | Raw Inst Captured: 00000000
Decode | Opcode: 0000000 | rs1 Data: 0 | rs2 Data: 0 | Target rd: x0
Exec | Calculated ALU Result: 20
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x
-----
--- Time: 85000 ns ---
Fetch | PC: 20 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: 0000000 | rs1 Data: 0 | rs2 Data: 0 | Target rd: x0
Exec | Calculated ALU Result: x
WriteB | RegFile[x1]= 10 | RegFile[x2]= x | RegFile[x3]= x
-----
--- Time: 95000 ns ---
Fetch | PC: 24 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data: x | rs2 Data: x | Target rd: xx
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= 10 | RegFile[x2]= 20 | RegFile[x3]= x
-----
--- Time: 105000 ns ---
Fetch | PC: 28 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data: x | rs2 Data: x | Target rd: xx
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= 10 | RegFile[x2]= 20 | RegFile[x3]= x
-----
--- Time: 115000 ns ---
Fetch | PC: 32 | Raw Inst Captured: xxxxxxxx

```

```

Decode | Opcode: xxxxxxx | rs1 Data:      x | rs2 Data:      x | Target rd: xx
Exec   | Calculated ALU Result:      0
WriteB | RegFile[x1]=      10 | RegFile[x2]=      20 | RegFile[x3]=      x
-----
--- Time: 125000 ns ---
Fetch  | PC:      36 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxx | rs1 Data:      x | rs2 Data:      x | Target rd: xx
Exec   | Calculated ALU Result:      0
WriteB | RegFile[x1]=      10 | RegFile[x2]=      20 | RegFile[x3]=      x
-----
--- Time: 135000 ns ---
Fetch  | PC:      40 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxx | rs1 Data:      x | rs2 Data:      x | Target rd: xx
Exec   | Calculated ALU Result:      0
WriteB | RegFile[x1]=      10 | RegFile[x2]=      20 | RegFile[x3]=      x
-----
--- Time: 145000 ns ---
Fetch  | PC:      44 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxx | rs1 Data:      x | rs2 Data:      x | Target rd: xx
Exec   | Calculated ALU Result:      0
WriteB | RegFile[x1]=      10 | RegFile[x2]=      20 | RegFile[x3]=      x
-----
--- Time: 155000 ns ---
Fetch  | PC:      48 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxx | rs1 Data:      x | rs2 Data:      x | Target rd: xx
Exec   | Calculated ALU Result:      0
WriteB | RegFile[x1]=      10 | RegFile[x2]=      20 | RegFile[x3]=      x
-----
--- Time: 165000 ns ---
Fetch  | PC:      52 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxx | rs1 Data:      x | rs2 Data:      x | Target rd: xx
Exec   | Calculated ALU Result:      0
WriteB | RegFile[x1]=      10 | RegFile[x2]=      20 | RegFile[x3]=      x
-----

```

Solving the RAW Hazard (Read After Write) using NOP

```

`timescale 1ns / 1ps
module tb_pipelined_processor();
    // Testbench Inputs (Regs)
    reg clk;
    reg rst;
    // Instantiate the Unit Under Test (UUT)
    pipelined_processor uut (
        .clk(clk),
        .rst(rst)
    );
    // VCD Dump configuration for EPWave visualization
    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_pipelined_processor);
    end

    // 1. Clock Generation (10ns period -> 100MHz clock)
    always begin
        #5 clk = ~clk;
    end
    // 2. Main Simulation Sequence
    initial begin
        // Initialize inputs
        clk = 0;
        rst = 1;
        // Wait 20ns for a stable system startup reset condition
        #20;
        // --- INSTRUCTION 1: addi x1, x0, 10 ---
    end

```

```

// Machine Code Hex: 32'h00A00093
uut.Inst_memory[0] = 8'h93;
uut.Inst_memory[1] = 8'h00;
uut.Inst_memory[2] = 8'hA0;
uut.Inst_memory[3] = 8'h00;
// --- INSTRUCTION 2: addi x2, x0, 20 ---
// Machine Code Hex: 32'h01400113
uut.Inst_memory[4] = 8'h13;
uut.Inst_memory[5] = 8'h01;
uut.Inst_memory[6] = 8'h40;
uut.Inst_memory[7] = 8'h01;
// Initialize trailing memory locations with NOP instructions (all zeros)
uut.Inst_memory[8] = 8'h00; uut.Inst_memory[9] = 8'h00; uut.Inst_memory[10] = 8'h00; uut.Inst_memory[11] =
8'h00;
uut.Inst_memory[12] = 8'h00; uut.Inst_memory[13] = 8'h00; uut.Inst_memory[14] = 8'h00; uut.Inst_memory[15]
= 8'h00;
uut.Inst_memory[16] = 8'h00; uut.Inst_memory[17] = 8'h00; uut.Inst_memory[18] = 8'h00; uut.Inst_memory[19] =
8'h00;
// --- INSTRUCTION 3: add x3, x1, x2 ---
// Machine Code Hex: 32'h002081b3
// Stored in Little-Endian starting at Address 8
uut.Inst_memory[20] = 8'hb3;
uut.Inst_memory[21] = 8'h81;
uut.Inst_memory[22] = 8'h20;
uut.Inst_memory[23] = 8'h00;
// Release reset to start execution
@(posedge clk);
#1 rst = 0;
// Running for 14 clock cycles to observe the hazard manifestation
repeat (14) @(posedge clk);
// End simulation safely
$finish;
end
// 3. Automated Runtime Monitoring Loop
always @(posedge clk) begin
if (!rst) begin
$display("--- Time: %0t ns ---", $time);
$display("Fetch | PC: %d | Raw Inst Captured: %h", uut.PC_register, uut.current_instruction);
$display("Decode | Opcode: %b | rs1 Data: %d | rs2 Data: %d | Target rd: x%0d",
uut.opcode, uut.reg_data1, uut.reg_data2, uut.rd);
$display("Exec | Calculated ALU Result: %d", uut.EXMEM_alu_result);
$display("WriteB | RegFile[x1]=%d | RegFile[x2]=%d | RegFile[x3]=%d",
uut.Register_File[1], uut.Register_File[2], uut.Register_File[3]);
$display("-----\n");
end
end
endmodule

```

clk												
rst												
PC_register[31:0]	NaN	0	4	8	12	16	20	24	28	32	36	40
current_instruction[31:0]	NaN	10485907	20971795	0			2130355	NaN				
IFID_instruction[31:0]	NaN	0	10485907	20971795	0			2130355	NaN			
rs1[4:0]	NaN	0						1	NaN			
rs2[4:0]	NaN	0	10	20	0			2	NaN			
reg_data1[31:0]	NaN	0						10	NaN			
reg_data2[31:0]	NaN	0	NaN		0			20	NaN			
IDEX_imm_ext[31:0]	NaN	0		10	20	0			2	NaN		
IDEX_reg_data1[31:0]	NaN	0							10	NaN		
IDEX_reg_data2[31:0]	NaN	0		NaN		0			20	NaN		
alu_result[31:0]	NaN	0		10	20	0			30	0		
EXMEM_alu_result[31:0]	NaN	0		10	20	0			30	0		
MEMWB_rd[4:0]	NaN	0				1	2	0		3	NaN	
MEMWB_alu_result[31:0]	NaN	0			10	20	0			30	0	

--- Time: 35000 ns ---

Fetch | PC: 0 | Raw Inst Captured: 00a00093
Decode | Opcode: 0000000 | rs1 Data: 0 | rs2 Data: 0 | Target rd: x0
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x

--- Time: 45000 ns ---

Fetch | PC: 4 | Raw Inst Captured: 01400113
Decode | Opcode: 0010011 | rs1 Data: 0 | rs2 Data: x | Target rd: x1
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x

--- Time: 55000 ns ---

Fetch | PC: 8 | Raw Inst Captured: 00000000
Decode | Opcode: 0010011 | rs1 Data: 0 | rs2 Data: x | Target rd: x2
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x

--- Time: 65000 ns ---

Fetch | PC: 12 | Raw Inst Captured: 00000000
Decode | Opcode: 0000000 | rs1 Data: 0 | rs2 Data: 0 | Target rd: x0
Exec | Calculated ALU Result: 10
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x

--- Time: 75000 ns ---

Fetch | PC: 16 | Raw Inst Captured: 00000000
Decode | Opcode: 0000000 | rs1 Data: 0 | rs2 Data: 0 | Target rd: x0
Exec | Calculated ALU Result: 20
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x

--- Time: 85000 ns ---

Fetch | PC: 20 | Raw Inst Captured: 002081b3
Decode | Opcode: 0000000 | rs1 Data: 0 | rs2 Data: 0 | Target rd: x0
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= 10 | RegFile[x2]= x | RegFile[x3]= x

--- Time: 95000 ns ---

Fetch | PC: 24 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: 0110011 | rs1 Data: 10 | rs2 Data: 20 | Target rd: x3
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= 10 | RegFile[x2]= 20 | RegFile[x3]= x

--- Time: 105000 ns ---

Fetch | PC: 28 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data: x | rs2 Data: x | Target rd: xx
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= 10 | RegFile[x2]= 20 | RegFile[x3]= x

--- Time: 115000 ns ---

Fetch | PC: 32 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data: x | rs2 Data: x | Target rd: xx
Exec | Calculated ALU Result: 30
WriteB | RegFile[x1]= 10 | RegFile[x2]= 20 | RegFile[x3]= x

--- Time: 125000 ns ---

Fetch | PC: 36 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data: x | rs2 Data: x | Target rd: xx
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= 10 | RegFile[x2]= 20 | RegFile[x3]= x

--- Time: 135000 ns ---

Fetch | PC: 40 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data: x | rs2 Data: x | Target rd: xx
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= 10 | RegFile[x2]= 20 | RegFile[x3]= 30

--- Time: 145000 ns ---

Fetch | PC: 44 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data: x | rs2 Data: x | Target rd: xx
Exec | Calculated ALU Result: 0

```

WriteB | RegFile[x1]=          10 | RegFile[x2]=          20 | RegFile[x3]=          30
-----
--- Time: 155000 ns ---
Fetch  | PC:          48 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data:          x | rs2 Data:          x | Target rd: xx
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1]=          10 | RegFile[x2]=          20 | RegFile[x3]=          30
-----
--- Time: 165000 ns ---
Fetch  | PC:          52 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data:          x | rs2 Data:          x | Target rd: xx
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1]=          10 | RegFile[x2]=          20 | RegFile[x3]=          30
-----

```

Solving the RAW Hazard (Read After Write) using Forwarding

```

module pipelined_processor(
    input clk,
    input rst
);
    wire [31:0] Nxt_PC;
    reg [31:0] PC_register;
    // Memory array: 256 bytes total
    reg [7:0] Inst_memory [0:255];
    // Wire to hold the raw fetched instruction from memory
    wire [31:0] current_instruction;
    // IF/ID Pipeline Registers
    reg [31:0] IFID_instruction;
    reg [31:0] IFID_PC;
    // ID/EX Pipeline Registers
    reg [31:0] IDEX_reg_data1;
    reg [31:0] IDEX_reg_data2;
    reg [4:0] IDEX_rs1;
    reg [4:0] IDEX_rs2;
    reg [4:0] IDEX_rd;
    reg [2:0] IDEX_funct3;
    reg [6:0] IDEX_funct7;
    reg [6:0] IDEX_opcode;
    // EX/MEM Pipeline Registers
    reg [31:0] EXMEM_alu_result;
    reg [4:0] EXMEM_rd;
    // MEM/WB Pipeline Registers
    reg [31:0] MEMWB_alu_result;
    reg [4:0] MEMWB_rd;
    // Operand for R-type
    wire [6:0] opcode = IFID_instruction[6:0];
    wire [4:0] rd = IFID_instruction[11:7];
    wire [2:0] funct3 = IFID_instruction[14:12];
    wire [4:0] rs1 = IFID_instruction[19:15];
    wire [4:0] rs2 = IFID_instruction[24:20];
    wire [6:0] funct7 = IFID_instruction[31:25];
    // Operand for I-type
    wire [31:0] imm_ext = { {20{IFID_instruction[31]}}, IFID_instruction[31:20] };
    // Register file
    reg [31:0] Register_File [0:31];
    wire [31:0] reg_data1;
    wire [31:0] reg_data2;
    // If rs1 is 0, output 0; otherwise, read from the register file array
    assign reg_data1 = (rs1 == 5'b0) ? 32'h0 : Register_File[rs1];
    assign reg_data2 = (rs2 == 5'b0) ? 32'h0 : Register_File[rs2];

```

```

// ALU Result
reg [31:0] alu_result;
reg [31:0] IDEX_imm_ext;
// Combinational PC Adder (PC + 4)
assign Nxt_PC = PC_register + 4;

// Sequential PC Register Update
always @(posedge clk) begin
    if (rst)
        PC_register <= 32'h0;    // Reset PC to address 0
    else
        PC_register <= Nxt_PC;    // Move to next sequential address
    end
// Combinational Instruction Fetch from Byte-Addressable Memory
assign current_instruction = {Inst_memory[PC_register + 3], Inst_memory[PC_register + 2],
Inst_memory[PC_register + 1], Inst_memory[PC_register] };

always @(posedge clk) begin
    if (rst) begin
        IFID_instruction <= 32'h0; // Flush instruction on reset (NOP)
        IFID_PC <= 32'h0;
    end else begin
        IFID_instruction <= current_instruction; // Pass instruction to next stage
        IFID_PC <= PC_register;    // Pass current instruction's PC
    end
end
always @(posedge clk) begin
    if (rst) begin
        IDEX_reg_data1 <= 32'h0;
        IDEX_reg_data2 <= 32'h0;
        IDEX_imm_ext <= 32'h0;
        IDEX_rd <= 5'h0;
        IDEX_funct3 <= 3'h0;
        IDEX_funct7 <= 7'h0;
        IDEX_opcode <= 7'h0;
    end else begin
        IDEX_reg_data1 <= reg_data1;
        IDEX_reg_data2 <= reg_data2;
        IDEX_imm_ext <= imm_ext;
        IDEX_rd <= rd;
        IDEX_funct3 <= funct3;
        IDEX_funct7 <= funct7;
        IDEX_opcode <= opcode; // Passing opcode downstream safely!
        IDEX_rs1 <= rs1;
        IDEX_rs2 <= rs2;
    end end
// Forwarding variables
reg [31:0] forwarded_val1;
reg [31:0] forwarded_val2;

always @* begin
    // Case 1: EX Hazard (Prior instruction in MEM stage is writing to our source register)
    if (EXMEM_rd != 5'b0 && EXMEM_rd == IDEX_rs1) begin
        forwarded_val1 = EXMEM_alu_result; // Grab directly from EX/MEM register
    end
    // Case 2: MEM Hazard (Older instruction in WB stage is writing to our source register)
    else if (MEMWB_rd != 5'b0 && MEMWB_rd == IDEX_rs1) begin

```

```

    forwarded_val1 = MEMWB_alu_result; // Grab directly from MEM/WB register
end
// Case 3: No Hazard
else begin
    forwarded_val1 = IDEX_reg_data1; // Use normal value from Register File
end
end
always @* begin
    // Case 1: EX Hazard (Prior instruction in MEM stage is writing to our source register)
    if (EXMEM_rd != 5'b0 && EXMEM_rd == IDEX_rs2) begin
        forwarded_val2 = EXMEM_alu_result; // Grab directly from EX/MEM register
    end
    // Case 2: MEM Hazard (Older instruction in WB stage is writing to our source register)
    else if (MEMWB_rd != 5'b0 && MEMWB_rd == IDEX_rs2) begin
        forwarded_val2 = MEMWB_alu_result; // Grab directly from MEM/WB register
    end
    // Case 3: No Hazard
    else begin
        forwarded_val2 = IDEX_reg_data2; // Use normal value from Register File
    end
end
end

wire [31:0] alu_operand2;
assign alu_operand2 = (IDEX_opcode == 7'b0010011) ? IDEX_imm_ext : forwarded_val2;

always @* begin
    alu_result = 32'h0;
    if (IDEX_opcode == 7'b0110011 && IDEX_func3 == 3'b000 && IDEX_func7 == 7'b0000000) begin
        alu_result = forwarded_val1 + alu_operand2;
    end
    else if (IDEX_opcode == 7'b0010011 && IDEX_func3 == 3'b000) begin
        alu_result = forwarded_val1 + alu_operand2;
    end
end
end

always @(posedge clk) begin
    if (rst) begin
        EXMEM_alu_result <= 32'h0;
        EXMEM_rd <= 5'h0;
    end else begin
        EXMEM_alu_result <= alu_result;
        EXMEM_rd <= IDEX_rd;
    end
end
end

always @(posedge clk) begin
    if (rst) begin
        MEMWB_alu_result <= 32'h0;
        MEMWB_rd <= 5'h0; end
    else begin
        MEMWB_alu_result <= EXMEM_alu_result;
        MEMWB_rd <= EXMEM_rd;
    end
end
end

always @(posedge clk) begin
    if (!rst && MEMWB_rd != 5'b0) begin
        Register_File[MEMWB_rd] <= MEMWB_alu_result;
    end
end
end

```

endmodule

clk										
rst										
PC_register[31:0]	NaN	0		4	8	12	16	20	24	28
current_instruction[31:0]	NaN		10485907	20971795	2130355	0		NaN		
IFID_instruction[31:0]	NaN	0	10485907	20971795	2130355	0		NaN		
rs1[4:0]	NaN	0			1	0			NaN	
rs2[4:0]	NaN	0	10	20	2	0			NaN	
reg_data1[31:0]	NaN	0			NaN	0			NaN	
reg_data2[31:0]	NaN	0	NaN		0				NaN	
IDEX_reg_data1[31:0]	NaN	0			NaN	0			NaN	
IDEX_reg_data2[31:0]	NaN	0		NaN		0			NaN	
alu_result[31:0]	NaN	0		10	20	30	0			
EXMEM_alu_result[31:0]	NaN	0			10	20	30	0		
MEMWB_rd[4:0]	NaN	0				1	2	3	0	
MEMWB_alu_result[31:0]	NaN	0				10	20	30	0	

--- Time: 35000 ns ---

```
Fetch | PC: 0 | Raw Inst Captured: 00a00093
Decode | Opcode: 0000000 | rs1 Data: 0 | rs2 Data: 0 | Target rd: x0
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x
```

--- Time: 45000 ns ---

```
Fetch | PC: 4 | Raw Inst Captured: 01400113
Decode | Opcode: 0010011 | rs1 Data: 0 | rs2 Data: x | Target rd: x1
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x
```

--- Time: 55000 ns ---

```
Fetch | PC: 8 | Raw Inst Captured: 002081b3
Decode | Opcode: 0010011 | rs1 Data: 0 | rs2 Data: x | Target rd: x2
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x
```

--- Time: 65000 ns ---

```
Fetch | PC: 12 | Raw Inst Captured: 00000000
Decode | Opcode: 0110011 | rs1 Data: x | rs2 Data: x | Target rd: x3
Exec | Calculated ALU Result: 10
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x
```

--- Time: 75000 ns ---

```
Fetch | PC: 16 | Raw Inst Captured: 00000000
Decode | Opcode: 0000000 | rs1 Data: 0 | rs2 Data: 0 | Target rd: x0
Exec | Calculated ALU Result: 20
WriteB | RegFile[x1]= x | RegFile[x2]= x | RegFile[x3]= x
```

--- Time: 85000 ns ---

```
Fetch | PC: 20 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: 0000000 | rs1 Data: 0 | rs2 Data: 0 | Target rd: x0
Exec | Calculated ALU Result: 30
WriteB | RegFile[x1]= 10 | RegFile[x2]= x | RegFile[x3]= x
```

--- Time: 95000 ns ---

```
Fetch | PC: 24 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data: x | rs2 Data: x | Target rd: xx
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= 10 | RegFile[x2]= 20 | RegFile[x3]= x
```

--- Time: 105000 ns ---

```
Fetch | PC: 28 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data: x | rs2 Data: x | Target rd: xx
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= 10 | RegFile[x2]= 20 | RegFile[x3]= 30
```

--- Time: 115000 ns ---

```
Fetch | PC: 32 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data: x | rs2 Data: x | Target rd: xx
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= 10 | RegFile[x2]= 20 | RegFile[x3]= 30
```

--- Time: 125000 ns ---

```
Fetch | PC: 36 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data: x | rs2 Data: x | Target rd: xx
Exec | Calculated ALU Result: 0
WriteB | RegFile[x1]= 10 | RegFile[x2]= 20 | RegFile[x3]= 30
```

--- Time: 135000 ns ---

```
Fetch | PC: 40 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data: x | rs2 Data: x | Target rd: xx
Exec | Calculated ALU Result: 0
```

```

WriteB | RegFile[x1]=          10 | RegFile[x2]=          20 | RegFile[x3]=          30
-----
--- Time: 145000 ns ---
Fetch  | PC:          44 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data:          x | rs2 Data:          x | Target rd: xx
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1]=          10 | RegFile[x2]=          20 | RegFile[x3]=          30
-----
--- Time: 155000 ns ---
Fetch  | PC:          48 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data:          x | rs2 Data:          x | Target rd: xx
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1]=          10 | RegFile[x2]=          20 | RegFile[x3]=          30
-----
--- Time: 165000 ns ---
Fetch  | PC:          52 | Raw Inst Captured: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1 Data:          x | rs2 Data:          x | Target rd: xx
Exec   | Calculated ALU Result:          0
WriteB | RegFile[x1]=          10 | RegFile[x2]=          20 | RegFile[x3]=          30
-----

```

Pipelined Processor with Load-Store, Add and Addi

```

module pipelined_processor(
    input clk,
    input rst
);

// =====
// 1. SIGNAL & ARCHITECTURAL REGISTERS DECLARATIONS
// =====

wire [31:0] Nxt_PC;
reg [31:0] PC_register;
// Memory Sub-systems
reg [7:0] Inst_memory [0:255];
reg [7:0] Data_memory [0:255];
// Core Decoding & Operational Wires
wire [31:0] current_instruction;
reg MemRead, MemWrite;
reg RegWrite, MemtoReg;
reg [31:0] alu_result;
wire [31:0] mem_read_data;
wire [31:0] final_write_back_data;
// Register File Storage & Ports
reg [31:0] Register_File [0:31];
wire [31:0] reg_data1;
wire [31:0] reg_data2;
// -----
// PIPELINE REGISTER FIELDS (Stage Boundary Latches)
// -----
// IF/ID Boundary Registers
reg [31:0] IFID_instruction;
reg [31:0] IFID_PC;

// ID/EX Boundary Registers
reg [31:0] IDEX_reg_data1;
reg [31:0] IDEX_reg_data2;
reg [31:0] IDEX_imm_ext;
reg [4:0] IDEX_rs1;
reg [4:0] IDEX_rs2;
reg [4:0] IDEX_rd;
reg [2:0] IDEX_funct3;
reg [6:0] IDEX_funct7;
reg [6:0] IDEX_opcode;

```

```
reg    IDEX_MemRead;
reg    IDEX_MemWrite;
reg    IDEX_RegWrite;
reg    IDEX_MemtoReg;
```

```
// EX/MEM Boundary Registers
```

```
reg [31:0] EXMEM_alu_result;
reg [31:0] EXMEM_reg_data2;
reg [4:0]  EXMEM_rd;
reg [6:0]  EXMEM_opcode;
reg    EXMEM_MemRead;
reg    EXMEM_MemWrite;
reg    EXMEM_RegWrite;
reg    EXMEM_MemtoReg;
```

```
// MEM/WB Boundary Registers
```

```
reg [31:0] MEMWB_alu_result;
reg [31:0] MEMWB_mem_read_data;
reg [4:0]  MEMWB_rd;
reg [6:0]  MEMWB_opcode;
reg    MEMWB_RegWrite;
reg    MEMWB_MemtoReg;
```

```
// =====
// STAGE 1: INSTRUCTION FETCH (IF)
// =====
```

```
// Combinational Next PC Evaluation Loop
```

```
assign Nxt_PC = PC_register + 4;
```

```
// Synchronous PC Program Register State Machine
```

```
always @(posedge clk) begin
```

```
    if (rst)
```

```
        PC_register <= 32'h0;
```

```
    else
```

```
        PC_register <= Nxt_PC;
```

```
end
```

```
// Combinational Little-Endian Byte Stitching from Program ROM
```

```
assign current_instruction = { Inst_memory[PC_register + 3],
                               Inst_memory[PC_register + 2],
                               Inst_memory[PC_register + 1],
                               Inst_memory[PC_register] };
```

```
// Synchronous Pipeline Intermediary: IF/ID
```

```
always @(posedge clk) begin
```

```
    if (rst) begin
```

```
        IFID_instruction <= 32'h0;
```

```
        IFID_PC         <= 32'h0;
```

```
    end else begin
```

```
        IFID_instruction <= current_instruction;
```

```
        IFID_PC         <= PC_register;
```

```
    end
```

```
end
```

```
// =====
// STAGE 2: INSTRUCTION DECODE / OPERAND FETCH (ID)
```

```
// =====
```

```
// Combinational Field Extraction Maps
```

```
wire [6:0] opcode = IFID_instruction[6:0];  
wire [4:0] rd    = IFID_instruction[11:7];  
wire [2:0] funct3 = IFID_instruction[14:12];  
wire [4:0] rs1   = IFID_instruction[19:15];  
wire [4:0] rs2   = IFID_instruction[24:20];  
wire [6:0] funct7 = IFID_instruction[31:25];
```

```
// Asymmetric Immediate Generator Logic (Supports both I-Type and S-Type layouts)
```

```
reg [31:0] imm_ext;  
always @(*) begin  
    if (opcode == 7'b0100011) begin  
        // S-type instruction layout (sw): Split bits [31:25] and [11:7]  
        imm_ext = { {20{IFID_instruction[31]}}, IFID_instruction[31:25], IFID_instruction[11:7] };  
    end else begin  
        // Standard I-type instruction layout (addi, lw): Straight bits [31:20]  
        imm_ext = { {20{IFID_instruction[31]}}, IFID_instruction[31:20] };  
    end  
end
```

```
// Register File Asynchronous Dual Read Interface (With Zero Hardwiring)
```

```
assign reg_data1 = (rs1 == 5'b0) ? 32'h0 : Register_File[rs1];  
assign reg_data2 = (rs2 == 5'b0) ? 32'h0 : Register_File[rs2];
```

```
// Combinational Main Control Decoder (Latch-Free Design)
```

```
always @(*) begin  
    MemRead  = 1'b0;  
    MemWrite = 1'b0;  
    RegWrite = 1'b0;  
    MemtoReg = 1'b0;  
  
    case (opcode)  
        7'b0110011: begin // R-type (add)  
            RegWrite = 1'b1;  
            MemtoReg = 1'b0;  
        end  
        7'b0010011: begin // I-type arithmetic (addi)  
            RegWrite = 1'b1;  
            MemtoReg = 1'b0;  
        end  
        7'b0000011: begin // lw (Load Word)  
            MemRead  = 1'b1;  
            RegWrite = 1'b1;  
            MemtoReg = 1'b1;  
        end  
        7'b0100011: begin // sw (Store Word)  
            MemWrite = 1'b1;  
            RegWrite = 1'b0;  
            MemtoReg = 1'b0;  
        end  
    endcase  
end
```

```
// Synchronous Pipeline Intermediary: ID/EX
```

```
always @(posedge clk) begin
```

```

if (rst) begin
    IDEX_reg_data1 <= 32'h0;
    IDEX_reg_data2 <= 32'h0;
    IDEX_imm_ext  <= 32'h0;
    IDEX_rd      <= 5'h0;
    IDEX_func3   <= 3'h0;
    IDEX_func7   <= 7'h0;
    IDEX_opcode  <= 7'h0;
    IDEX_rs1     <= 5'h0;
    IDEX_rs2     <= 5'h0;
    IDEX_MemRead <= 1'b0;
    IDEX_MemWrite <= 1'b0;
    IDEX_RegWrite <= 1'b0;
    IDEX_MemtoReg <= 1'b0;
end else begin
    IDEX_reg_data1 <= reg_data1;
    IDEX_reg_data2 <= reg_data2;
    IDEX_imm_ext  <= imm_ext;
    IDEX_rd      <= rd;
    IDEX_func3   <= funct3;
    IDEX_func7   <= funct7;
    IDEX_opcode  <= opcode;
    IDEX_rs1     <= rs1;
    IDEX_rs2     <= rs2;
    IDEX_MemRead <= MemRead;
    IDEX_MemWrite <= MemWrite;
    IDEX_RegWrite <= RegWrite;
    IDEX_MemtoReg <= MemtoReg;
end
end

// =====
// STAGE 3: EXECUTE / ADDRESS GENERATION (EX)
// =====

reg [31:0] forwarded_val1;
reg [31:0] forwarded_val2;
wire [31:0] alu_operand2;

// Operand 1 Dynamic Bypassing / Forwarding Matrix Multiplexer
always @(*) begin
    if (EXMEM_rd != 5'b0 && EXMEM_rd == IDEX_rs1 && EXMEM_RegWrite) begin
        forwarded_val1 = EXMEM_alu_result; // EX-to-EX Hazard Rescue
    end else if (MEMWB_rd != 5'b0 && MEMWB_rd == IDEX_rs1 && MEMWB_RegWrite) begin
        forwarded_val1 = final_write_back_data; // MEM-to-EX Hazard Rescue
    end else begin
        forwarded_val1 = IDEX_reg_data1; // Standard Path
    end
end

// Operand 2 Dynamic Bypassing / Forwarding Matrix Multiplexer
always @(*) begin
    if (EXMEM_rd != 5'b0 && EXMEM_rd == IDEX_rs2 && EXMEM_RegWrite) begin
        forwarded_val2 = EXMEM_alu_result; // EX-to-EX Hazard Rescue
    end else if (MEMWB_rd != 5'b0 && MEMWB_rd == IDEX_rs2 && MEMWB_RegWrite) begin
        forwarded_val2 = final_write_back_data; // MEM-to-EX Hazard Rescue
    end else begin

```

```

        forwarded_val2 = IDEX_reg_data2; // Standard Path
    end
end

// ALU Secondary Operand Source Ingestion Selector Multiplexer
assign alu_operand2 = (IDEX_opcode == 7'b0010011 || IDEX_opcode == 7'b0000011 || IDEX_opcode ==
7'b0100011) ? IDEX_imm_ext : forwarded_val2;

// Mathematical ALU Calculation Engine
always @(*) begin
    alu_result = 32'h0;
    if (IDEX_opcode == 7'b0110011 && IDEX_func3 == 3'b000 && IDEX_func7 == 7'b0000000) begin
        alu_result = forwarded_val1 + alu_operand2; // add
    end else if (IDEX_opcode == 7'b0010011 && IDEX_func3 == 3'b000) begin
        alu_result = forwarded_val1 + alu_operand2; // addi
    end else if (IDEX_opcode == 7'b0000011 || IDEX_opcode == 7'b0100011) begin
        alu_result = forwarded_val1 + alu_operand2; // lw / sw Address Generation (Base + Offset)
    end
end

// Synchronous Pipeline Intermediary: EX/MEM
always @(posedge clk) begin
    if (rst) begin
        EXMEM_alu_result <= 32'h0;
        EXMEM_reg_data2 <= 32'h0;
        EXMEM_rd <= 5'h0;
        EXMEM_opcode <= 7'h0;
        EXMEM_MemRead <= 1'b0;
        EXMEM_MemWrite <= 1'b0;
        EXMEM_RegWrite <= 1'b0;
        EXMEM_MemtoReg <= 1'b0;
    end else begin
        EXMEM_alu_result <= alu_result;
        EXMEM_reg_data2 <= forwarded_val2; // Secure the forwarded store register payload
        EXMEM_rd <= IDEX_rd;
        EXMEM_opcode <= IDEX_opcode;
        EXMEM_MemRead <= IDEX_MemRead;
        EXMEM_MemWrite <= IDEX_MemWrite;
        EXMEM_RegWrite <= IDEX_RegWrite;
        EXMEM_MemtoReg <= IDEX_MemtoReg;
    end
end

// =====
// STAGE 4: DATA MEMORY ACCESS (MEM)
// =====

// Little-Endian Combinational Bus Word Stitching for Reads (RAM -> Wire)
assign mem_read_data = (EXMEM_MemRead) ? { Data_memory[EXMEM_alu_result + 3],
        Data_memory[EXMEM_alu_result + 2],
        Data_memory[EXMEM_alu_result + 1],
        Data_memory[EXMEM_alu_result] } : 32'h0;

// Little-Endian Synchronous Memory Array Splitting for Writes (Register -> RAM)
always @(posedge clk) begin
    if (EXMEM_MemWrite) begin
        Data_memory[EXMEM_alu_result] <= EXMEM_reg_data2[7:0];
    end
end

```

```

    Data_memory[EXMEM_alu_result + 1] <= EXMEM_reg_data2[15:8];
    Data_memory[EXMEM_alu_result + 2] <= EXMEM_reg_data2[23:16];
    Data_memory[EXMEM_alu_result + 3] <= EXMEM_reg_data2[31:24];
end
end

```

```

// Synchronous Pipeline Intermediary: MEM/WB

```

```

always @(posedge clk) begin
    if (rst) begin
        MEMWB_alu_result  <= 32'h0;
        MEMWB_mem_read_data <= 32'h0;
        MEMWB_rd          <= 5'h0;
        MEMWB_opcode       <= 7'h0;
        MEMWB_RegWrite     <= 1'b0;
        MEMWB_MemtoReg     <= 1'b0;
    end else begin
        MEMWB_alu_result  <= EXMEM_alu_result;
        MEMWB_mem_read_data <= mem_read_data;
        MEMWB_rd          <= EXMEM_rd;
        MEMWB_opcode       <= EXMEM_opcode;
        MEMWB_RegWrite     <= EXMEM_RegWrite;
        MEMWB_MemtoReg     <= EXMEM_MemtoReg;
    end
end
end

```

```

// =====
// STAGE 5: WRITE-BACK LOOP (WB)
// =====

```

```

// Architectural Committing Source Selection Multiplexer

```

```

assign final_write_back_data = (MEMWB_MemtoReg) ? MEMWB_mem_read_data : MEMWB_alu_result;

```

```

// Synchronous Architectural Register File Write Commit Loop

```

```

always @(posedge clk) begin
    if (!rst && MEMWB_RegWrite && MEMWB_rd != 5'h0) begin
        Register_File[MEMWB_rd] <= final_write_back_data;
    end
end
end

```

```

endmodule

```

```

`timescale 1ns / 1ps

```

```

module tb_pipelined_processor();

```

```

    // Testbench Control Inputs

```

```

    reg clk;
    reg rst;

```

```

    // Instantiate the Unified Pipelined Processor

```

```

    pipelined_processor uut (
        .clk(clk),
        .rst(rst)
    );

```

```

    // VCD Waveform Generation Configuration for EPWave

```

```

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_pipelined_processor);
end

// 1. Clock Generation (10ns Period -> 100MHz Simulation Frequency)
always begin
    #5 clk = ~clk;
end

// 2. Main Test Trace Simulation Sequence
// 1. Separate Memory Initialization Block (Executes first)
initial begin
    // --- INSTRUCTION 1: addi x1, x0, 85 --- (Hex: 32'h05500093)
    uut.Inst_memory[0] = 8'h93;
    uut.Inst_memory[1] = 8'h00;
    uut.Inst_memory[2] = 8'h50;
    uut.Inst_memory[3] = 8'h05;

    // --- INSTRUCTION 2: sw x1, 32(x0) --- (Hex: 32'h02102023)
    uut.Inst_memory[4] = 8'h23;
    uut.Inst_memory[5] = 8'h20;
    uut.Inst_memory[6] = 8'h10;
    uut.Inst_memory[7] = 8'h02;

    // --- INSTRUCTION 3: lw x4, 32(x0) --- (Hex: 32'h02002203)
    uut.Inst_memory[8] = 8'h03;
    uut.Inst_memory[9] = 8'h22;
    uut.Inst_memory[10] = 8'h00;
    uut.Inst_memory[11] = 8'h02;

    // --- TRAILING FLUSH PADDING (NOP Instructions) ---
    uut.Inst_memory[12] = 8'h00; uut.Inst_memory[13] = 8'h00; uut.Inst_memory[14] = 8'h00;
    uut.Inst_memory[15] = 8'h00;
    uut.Inst_memory[16] = 8'h00; uut.Inst_memory[17] = 8'h00; uut.Inst_memory[18] = 8'h00;
    uut.Inst_memory[19] = 8'h00;
    uut.Inst_memory[20] = 8'h00; uut.Inst_memory[21] = 8'h00; uut.Inst_memory[22] = 8'h00;
    uut.Inst_memory[23] = 8'h00;
end

// 2. Main Simulation & Control Sequence
initial begin
    // Initialize inputs
    clk = 0;
    rst = 1;

    // Hold reset for 20ns to let the stable system start up safely
    #20;

    // Release reset synchronously on a clock edge
    @(posedge clk);
    #1 rst = 0;

    // Run simulation for 14 cycles to observe the load/store path completion
    repeat (14) @(posedge clk);

    // Close out simulation safely

```

```

$finish;
end

// 3. Hardware Monitoring Trace Loop
always @(posedge clk) begin
    if (!rst) begin
        $display("--- Time: %0t ns ---", $time);
        $display("Fetch | PC: %d | Raw Inst: %h", uut.PC_register, uut.current_instruction);
        $display("Decode | Opcode: %b | rs1: x%0d | rs2: x%0d | rd: x%0d",
            uut.IDEX_opcode, uut.IDEX_rs1, uut.IDEX_rs2, uut.IDEX_rd);
        $display("Exec | Calculated Address/ALU Result: %d", uut.EXMEM_alu_result);
        $display("Memory | MemRead En: %b | MemWrite En: %b | Raw RAM Output Data: %d",
            uut.EXMEM_MemRead, uut.EXMEM_MemWrite, uut.mem_read_data);
        $display("Status | RAM[32]=%d | RegFile[x1]=%d | RegFile[x4]=%d",
            {uut.Data_memory[35], uut.Data_memory[34], uut.Data_memory[33], uut.Data_memory[32]},
            uut.Register_File[1], uut.Register_File[4]);
        $display("-----\n");
    end
end
end

```

endmodule

--- Time: 35000 ns ---

```

Fetch | PC:      0 | Raw Inst: 05500093
Decode | Opcode: 0000000 | rs1: x0 | rs2: x0 | rd: x0
Exec | Calculated Address/ALU Result:      0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Output Data:      0
Status | RAM[32]=    x | RegFile[x1]=    x | RegFile[x4]=    x
-----

```

--- Time: 45000 ns ---

```

Fetch | PC:      4 | Raw Inst: 02102023
Decode | Opcode: 0000000 | rs1: x0 | rs2: x0 | rd: x0
Exec | Calculated Address/ALU Result:      0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Output Data:      0
Status | RAM[32]=    x | RegFile[x1]=    x | RegFile[x4]=    x
-----

```

--- Time: 55000 ns ---

```

Fetch | PC:      8 | Raw Inst: 02002203
Decode | Opcode: 0010011 | rs1: x0 | rs2: x21 | rd: x1
Exec | Calculated Address/ALU Result:      0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Output Data:      0
Status | RAM[32]=    x | RegFile[x1]=    x | RegFile[x4]=    x
-----

```

--- Time: 65000 ns ---

```

Fetch | PC:     12 | Raw Inst: 00000000
Decode | Opcode: 0100011 | rs1: x0 | rs2: x1 | rd: x0
Exec | Calculated Address/ALU Result:     85
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Output Data:      0
Status | RAM[32]=    x | RegFile[x1]=    x | RegFile[x4]=    x
-----

```

--- Time: 75000 ns ---

```

Fetch | PC:     16 | Raw Inst: 00000000
Decode | Opcode: 0000011 | rs1: x0 | rs2: x0 | rd: x4

```

```

Exec | Calculated Address/ALU Result:    32
Memory | MemRead En: 0 | MemWrite En: 1 | Raw RAM Output Data:    0
Status | RAM[32]=    x | RegFile[x1]=    x | RegFile[x4]=    x
-----

--- Time: 85000 ns ---
Fetch | PC:    20 | Raw Inst: 00000000
Decode | Opcode: 00000000 | rs1: x0 | rs2: x0 | rd: x0
Exec | Calculated Address/ALU Result:    32
Memory | MemRead En: 1 | MemWrite En: 0 | Raw RAM Output Data:    85
Status | RAM[32]=    85 | RegFile[x1]=    85 | RegFile[x4]=    x
-----

--- Time: 95000 ns ---
Fetch | PC:    24 | Raw Inst: xxxxxxxx
Decode | Opcode: 00000000 | rs1: x0 | rs2: x0 | rd: x0
Exec | Calculated Address/ALU Result:    0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Output Data:    0
Status | RAM[32]=    85 | RegFile[x1]=    85 | RegFile[x4]=    x
-----

--- Time: 105000 ns ---
Fetch | PC:    28 | Raw Inst: xxxxxxxx
Decode | Opcode: 00000000 | rs1: x0 | rs2: x0 | rd: x0
Exec | Calculated Address/ALU Result:    0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Output Data:    0
Status | RAM[32]=    85 | RegFile[x1]=    85 | RegFile[x4]=    85
-----

--- Time: 115000 ns ---
Fetch | PC:    32 | Raw Inst: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1: xx | rs2: xx | rd: xx
Exec | Calculated Address/ALU Result:    0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Output Data:    0
Status | RAM[32]=    85 | RegFile[x1]=    85 | RegFile[x4]=    85
-----

--- Time: 125000 ns ---
Fetch | PC:    36 | Raw Inst: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1: xx | rs2: xx | rd: xx
Exec | Calculated Address/ALU Result:    0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Output Data:    0
Status | RAM[32]=    85 | RegFile[x1]=    85 | RegFile[x4]=    85
-----

--- Time: 135000 ns ---
Fetch | PC:    40 | Raw Inst: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1: xx | rs2: xx | rd: xx
Exec | Calculated Address/ALU Result:    0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Output Data:    0
Status | RAM[32]=    85 | RegFile[x1]=    85 | RegFile[x4]=    85
-----

--- Time: 145000 ns ---
Fetch | PC:    44 | Raw Inst: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1: xx | rs2: xx | rd: xx
Exec | Calculated Address/ALU Result:    0

```

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Output Data: 0
Status | RAM[32]= 85 | RegFile[x1]= 85 | RegFile[x4]= 85

--- Time: 155000 ns ---

Fetch | PC: 48 | Raw Inst: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1: xx | rs2: xx | rd: xx
Exec | Calculated Address/ALU Result: 0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Output Data: 0
Status | RAM[32]= 85 | RegFile[x1]= 85 | RegFile[x4]= 85

--- Time: 165000 ns ---

Fetch | PC: 52 | Raw Inst: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1: xx | rs2: xx | rd: xx
Exec | Calculated Address/ALU Result: 0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Output Data: 0
Status | RAM[32]= 85 | RegFile[x1]= 85 | RegFile[x4]= 85

Pipelined Processor with Branching

```
module pipelined_processor(  
    input clk,  
    input rst  
);  
  
    // =====  
    // 1. SIGNAL & ARCHITECTURAL REGISTERS DECLARATIONS  
    // =====  
  
    wire [31:0] Nxt_PC;  
    reg [31:0] PC_register;  
  
    // Memory Sub-systems  
    reg [7:0] Inst_memory [0:255];  
    reg [7:0] Data_memory [0:255];  
  
    // Core Decoding & Operational Wires  
    wire [31:0] current_instruction;  
    reg MemRead, MemWrite;  
    reg RegWrite, MemtoReg;  
    reg [31:0] alu_result;  
    wire [31:0] mem_read_data;  
    wire [31:0] final_write_back_data;  
  
    // Control Hazard Resolution Line  
    wire PCSrc;  
  
    // Register File Storage & Ports  
    reg [31:0] Register_File [0:31];  
    wire [31:0] reg_data1;  
    wire [31:0] reg_data2;  
  
    // -----
```

```

// PIPELINE REGISTER FIELDS (Stage Boundary Latches)
// -----

// IF/ID Boundary Registers
reg [31:0] IFID_instruction;
reg [31:0] IFID_PC;

// ID/EX Boundary Registers
reg [31:0] IDEX_reg_data1;
reg [31:0] IDEX_reg_data2;
reg [31:0] IDEX_imm_ext;
reg [4:0] IDEX_rs1;
reg [4:0] IDEX_rs2;
reg [4:0] IDEX_rd;
reg [2:0] IDEX_funct3;
reg [6:0] IDEX_funct7;
reg [6:0] IDEX_opcode;
reg [31:0] IDEX_PC;      // Passed down to compute Branch Target in EX stage
reg IDEX_MemRead;
reg IDEX_MemWrite;
reg IDEX_RegWrite;
reg IDEX_MemtoReg;

// EX/MEM Boundary Registers
reg [31:0] EXMEM_alu_result;
reg [31:0] EXMEM_reg_data2;
reg [4:0] EXMEM_rd;
reg [6:0] EXMEM_opcode;
reg EXMEM_MemRead;
reg EXMEM_MemWrite;
reg EXMEM_RegWrite;
reg EXMEM_MemtoReg;

// MEM/WB Boundary Registers
reg [31:0] MEMWB_alu_result;
reg [31:0] MEMWB_mem_read_data;
reg [4:0] MEMWB_rd;
reg [6:0] MEMWB_opcode;
reg MEMWB_RegWrite;
reg MEMWB_MemtoReg;

// Asynchronous Hardware Power-On Reset Initialization Array Matrix Loop
integer i;
initial begin
    for (i = 0; i < 32; i = i + 1) begin
        Register_File[i] = 32'h0;
    end
end

// =====
// STAGE 1: INSTRUCTION FETCH (IF)
// =====

// Combinational Next PC Evaluation Loop (Chooses branch target if branch is taken)
assign Nxt_PC = (PCSrc) ? (IDEX_PC + IDEX_imm_ext) : (PC_register + 4);

// Synchronous PC Program Register State Machine

```

```

always @(posedge clk) begin
    if (rst)
        PC_register <= 32'h0;
    else
        PC_register <= Nxt_PC;
end

// Combinational Little-Endian Byte Stitching from Program ROM
assign current_instruction = { Inst_memory[PC_register + 3],
                               Inst_memory[PC_register + 2],
                               Inst_memory[PC_register + 1],
                               Inst_memory[PC_register] };

// Synchronous Pipeline Intermediary: IF/ID (With Control Hazard Flushing)
always @(posedge clk) begin
    if (rst || PCSrc) begin
        // Flushes the instruction in flight, turning it into a NOP (Bubble)
        IFID_instruction <= 32'h0;
        IFID_PC <= 32'h0;
    end else begin
        IFID_instruction <= current_instruction;
        IFID_PC <= PC_register;
    end
end

// =====
// STAGE 2: INSTRUCTION DECODE / OPERAND FETCH (ID)
// =====

// Combinational Field Extraction Maps
wire [6:0] opcode = IFID_instruction[6:0];
wire [4:0] rd = IFID_instruction[11:7];
wire [2:0] funct3 = IFID_instruction[14:12];
wire [4:0] rs1 = IFID_instruction[19:15];
wire [4:0] rs2 = IFID_instruction[24:20];
wire [6:0] funct7 = IFID_instruction[31:25];

// Asymmetric Immediate Generator Logic (Supports I-Type, S-Type, and B-Type Layouts)
reg [31:0] imm_ext;
always @(*) begin
    if (opcode == 7'b0100011) begin
        // S-type instruction layout (sw)
        imm_ext = { {20{IFID_instruction[31]}}, IFID_instruction[31:25], IFID_instruction[11:7] };
    end else if (opcode == 7'b1100011) begin
        // B-type instruction layout (beq): Conditional branch offsets are multi-split
        imm_ext = { {20{IFID_instruction[31]}}, IFID_instruction[7], IFID_instruction[30:25], IFID_instruction[11:8],
1'b0 };
    end else begin
        // Standard I-type instruction layout (addi, lw)
        imm_ext = { {20{IFID_instruction[31]}}, IFID_instruction[31:20] };
    end
end

// Register File Asynchronous Dual Read Interface (With Zero Hardwiring)
assign reg_data1 = (rs1 == 5'b0) ? 32'h0 : Register_File[rs1];
assign reg_data2 = (rs2 == 5'b0) ? 32'h0 : Register_File[rs2];

```

```
// Combinational Main Control Decoder (Latch-Free Design)
```

```
always @(*) begin
```

```
    MemRead = 1'b0;
```

```
    MemWrite = 1'b0;
```

```
    RegWrite = 1'b0;
```

```
    MemtoReg = 1'b0;
```

```
case (opcode)
```

```
    7'b0110011: begin // R-type (add)
```

```
        RegWrite = 1'b1;
```

```
        MemtoReg = 1'b0;
```

```
    end
```

```
    7'b0010011: begin // I-type arithmetic (addi)
```

```
        RegWrite = 1'b1;
```

```
        MemtoReg = 1'b0;
```

```
    end
```

```
    7'b0000011: begin // lw (Load Word)
```

```
        MemRead = 1'b1;
```

```
        RegWrite = 1'b1;
```

```
        MemtoReg = 1'b1;
```

```
    end
```

```
    7'b0100011: begin // sw (Store Word)
```

```
        MemWrite = 1'b1;
```

```
        RegWrite = 1'b0;
```

```
        MemtoReg = 1'b0;
```

```
    end
```

```
    7'b1100011: begin // beq (Branch if Equal)
```

```
        MemWrite = 1'b0;
```

```
        RegWrite = 1'b0;
```

```
        MemtoReg = 1'b0;
```

```
    end
```

```
endcase
```

```
end
```

```
// Synchronous Pipeline Intermediary: ID/EX (With Control Hazard Flushes)
```

```
always @(posedge clk) begin
```

```
    if (rst || PCSrc) begin // Flushes decoded registers if branch condition evaluates to true
```

```
        IDEX_reg_data1 <= 32'h0;
```

```
        IDEX_reg_data2 <= 32'h0;
```

```
        IDEX_imm_ext <= 32'h0;
```

```
        IDEX_rd <= 5'h0;
```

```
        IDEX_funct3 <= 3'h0;
```

```
        IDEX_funct7 <= 7'h0;
```

```
        IDEX_opcode <= 7'h0;
```

```
        IDEX_rs1 <= 5'h0;
```

```
        IDEX_rs2 <= 5'h0;
```

```
        IDEX_PC <= 32'h0;
```

```
        IDEX_MemRead <= 1'b0;
```

```
        IDEX_MemWrite <= 1'b0;
```

```
        IDEX_RegWrite <= 1'b0;
```

```
        IDEX_MemtoReg <= 1'b0;
```

```
    end else begin
```

```
        IDEX_reg_data1 <= reg_data1;
```

```
        IDEX_reg_data2 <= reg_data2;
```

```
        IDEX_imm_ext <= imm_ext;
```

```
        IDEX_rd <= rd;
```

```
        IDEX_funct3 <= funct3;
```

```

    IDEX_func7  <= funct7;
    IDEX_opcode <= opcode;
    IDEX_rs1    <= rs1;
    IDEX_rs2    <= rs2;
    IDEX_PC     <= IFID_PC;
    IDEX_MemRead <= MemRead;
    IDEX_MemWrite <= MemWrite;
    IDEX_RegWrite <= RegWrite;
    IDEX_MemtoReg <= MemtoReg;
end
end

// =====
// STAGE 3: EXECUTE / ADDRESS GENERATION (EX)
// =====

reg [31:0] forwarded_val1;
reg [31:0] forwarded_val2;
wire [31:0] alu_operand2;

// Operand 1 Dynamic Bypassing / Forwarding Matrix Multiplexer
always @(*) begin
    if (EXMEM_rd != 5'b0 && EXMEM_rd == IDEX_rs1 && EXMEM_RegWrite) begin
        forwarded_val1 = EXMEM_alu_result;
    end else if (MEMWB_rd != 5'b0 && MEMWB_rd == IDEX_rs1 && MEMWB_RegWrite) begin
        forwarded_val1 = final_write_back_data;
    end else begin
        forwarded_val1 = IDEX_reg_data1;
    end
end

// Operand 2 Dynamic Bypassing / Forwarding Matrix Multiplexer
always @(*) begin
    if (EXMEM_rd != 5'b0 && EXMEM_rd == IDEX_rs2 && EXMEM_RegWrite) begin
        forwarded_val2 = EXMEM_alu_result;
    end else if (MEMWB_rd != 5'b0 && MEMWB_rd == IDEX_rs2 && MEMWB_RegWrite) begin
        forwarded_val2 = final_write_back_data;
    end else begin
        forwarded_val2 = IDEX_reg_data2;
    end
end

// ALU Secondary Operand Source Ingestion Selector Multiplexer
assign alu_operand2 = (IDEX_opcode == 7'b0010011 || IDEX_opcode == 7'b0000011 || IDEX_opcode ==
7'b0100011) ? IDEX_imm_ext : forwarded_val2;

// Control Hazard Decision: Assert PCSrc if instruction is a branch AND values match
assign PCSrc = (IDEX_opcode == 7'b1100011) && (forwarded_val1 == forwarded_val2);

// Mathematical ALU Calculation Engine
always @(*) begin
    alu_result = 32'h0;
    if (IDEX_opcode == 7'b0110011 && IDEX_func3 == 3'b000 && IDEX_func7 == 7'b0000000) begin
        alu_result = forwarded_val1 + alu_operand2; // add
    end else if (IDEX_opcode == 7'b0010011 && IDEX_func3 == 3'b000) begin
        alu_result = forwarded_val1 + alu_operand2; // addi
    end else if (IDEX_opcode == 7'b0000011 || IDEX_opcode == 7'b0100011) begin

```

```

    alu_result = forwarded_val1 + alu_operand2; // lw / sw Address Generation
end
end

// Synchronous Pipeline Intermediary: EX/MEM
always @(posedge clk) begin
    if (rst) begin
        EXMEM_alu_result <= 32'h0;
        EXMEM_reg_data2 <= 32'h0;
        EXMEM_rd <= 5'h0;
        EXMEM_opcode <= 7'h0;
        EXMEM_MemRead <= 1'b0;
        EXMEM_MemWrite <= 1'b0;
        EXMEM_RegWrite <= 1'b0;
        EXMEM_MemtoReg <= 1'b0;
    end else begin
        EXMEM_alu_result <= alu_result;
        EXMEM_reg_data2 <= forwarded_val2;
        EXMEM_rd <= IDEX_rd;
        EXMEM_opcode <= IDEX_opcode;
        EXMEM_MemRead <= IDEX_MemRead;
        EXMEM_MemWrite <= IDEX_MemWrite;
        EXMEM_RegWrite <= IDEX_RegWrite;
        EXMEM_MemtoReg <= IDEX_MemtoReg;
    end
end

// =====
// STAGE 4: DATA MEMORY ACCESS (MEM)
// =====

// Little-Endian Combinational Bus Word Stitching for Reads (RAM -> Wire)
assign mem_read_data = (EXMEM_MemRead) ? { Data_memory[EXMEM_alu_result + 3],
                                           Data_memory[EXMEM_alu_result + 2],
                                           Data_memory[EXMEM_alu_result + 1],
                                           Data_memory[EXMEM_alu_result] } : 32'h0;

// Little-Endian Synchronous Memory Array Splitting for Writes (Register -> RAM)
always @(posedge clk) begin
    if (EXMEM_MemWrite) begin
        Data_memory[EXMEM_alu_result] <= EXMEM_reg_data2[7:0];
        Data_memory[EXMEM_alu_result + 1] <= EXMEM_reg_data2[15:8];
        Data_memory[EXMEM_alu_result + 2] <= EXMEM_reg_data2[23:16];
        Data_memory[EXMEM_alu_result + 3] <= EXMEM_reg_data2[31:24];
    end
end

// Synchronous Pipeline Intermediary: MEM/WB
always @(posedge clk) begin
    if (rst) begin
        MEMWB_alu_result <= 32'h0;
        MEMWB_mem_read_data <= 32'h0;
        MEMWB_rd <= 5'h0;
        MEMWB_opcode <= 7'h0;
        MEMWB_RegWrite <= 1'b0;
        MEMWB_MemtoReg <= 1'b0;
    end else begin

```

```

    MEMWB_alu_result  <= EXMEM_alu_result;
    MEMWB_mem_read_data <= mem_read_data;
    MEMWB_rd          <= EXMEM_rd;
    MEMWB_opcode      <= EXMEM_opcode;
    MEMWB_RegWrite    <= EXMEM_RegWrite;
    MEMWB_MemtoReg    <= EXMEM_MemtoReg;
end
end

// =====
// STAGE 5: WRITE-BACK LOOP (WB)
// =====

// Architectural Committing Source Selection Multiplexer
assign final_write_back_data = (MEMWB_MemtoReg) ? MEMWB_mem_read_data : MEMWB_alu_result;

// Synchronous Architectural Register File Write Commit Loop
always @(posedge clk) begin
    if (!rst && MEMWB_RegWrite && MEMWB_rd != 5'b0) begin
        Register_File[MEMWB_rd] <= final_write_back_data;
    end
end

endmodule

`timescale 1ns / 1ps

module tb_pipelined_processor();

    // Testbench Control Inputs
    reg clk;
    reg rst;

    // Instantiate the Unified Pipelined Processor
    pipelined_processor uut (
        .clk(clk),
        .rst(rst)
    );

    // VCD Waveform Generation Configuration for EPWave
    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_pipelined_processor);
    end

    // 1. Clock Generation (10ns Period -> 100MHz Simulation Frequency)
    always begin
        #5 clk = ~clk;
    end

    // =====
    // CORRECTED MACHINE CODE MAP: (Fixes Immediate Byte Shifts)
    // =====
    initial begin
        // --- INSTRUCTION 1: addi x1, x0, 55 --- (Correct Word: 32'h03700093)
        uut.Inst_memory[0] = 8'h93;
        uut.Inst_memory[1] = 8'h00;
    end

```

```

uut.Inst_memory[2] = 8'h70; // Lower half of immediate + upper rd bits
uut.Inst_memory[3] = 8'h03; // Upper immediate bits (Holds the 3)

// --- INSTRUCTION 2: addi x2, x0, 55 --- (Correct Word: 32'h03700113)
uut.Inst_memory[4] = 8'h13;
uut.Inst_memory[5] = 8'h01;
uut.Inst_memory[6] = 8'h70;
uut.Inst_memory[7] = 8'h03;

// --- INSTRUCTION 3: beq x1, x2, 8 --- (Correct Word: 32'h00208463)
uut.Inst_memory[8] = 8'h63;
uut.Inst_memory[9] = 8'h84;
uut.Inst_memory[10] = 8'h20;
uut.Inst_memory[11] = 8'h00;

// --- INSTRUCTION 4 (TRAP): addi x3, x0, 99 --- (Correct Word: 32'h06300193)
uut.Inst_memory[12] = 8'h93;
uut.Inst_memory[13] = 8'h01;
uut.Inst_memory[14] = 8'h30;
uut.Inst_memory[15] = 8'h06;

// --- INSTRUCTION 5 (TARGET): addi x4, x0, 22 --- (Correct Word: 32'h01600213)
uut.Inst_memory[16] = 8'h13;
uut.Inst_memory[17] = 8'h02;
uut.Inst_memory[18] = 8'h60;
uut.Inst_memory[19] = 8'h01;

// NOP Trailing Padding
uut.Inst_memory[20] = 8'h00; uut.Inst_memory[21] = 8'h00; uut.Inst_memory[22] = 8'h00;
uut.Inst_memory[23] = 8'h00;
uut.Inst_memory[24] = 8'h00; uut.Inst_memory[25] = 8'h00; uut.Inst_memory[26] = 8'h00;
uut.Inst_memory[27] = 8'h00;
end

// 3. Main Simulation Control Sequence
initial begin
    clk = 0;
    rst = 1;

    // Wait 20ns for stable startup reset state
    #20;

    // Release reset synchronously on a clock edge
    @(posedge clk);
    #1 rst = 0;

    // Run for 14 cycles to watch the branch evaluate, flush, and redirect
    repeat (14) @(posedge clk);

    // End simulation safely
    $finish;
end

always @(negedge clk) begin
    if (!rst) begin
        $display("--- Time: %0t ns (Falling Clock Edge) ---", $time);
        $display("Fetch | PC: %0d | Raw Inst: %h", uut.PC_register, uut.current_instruction);
    end
end

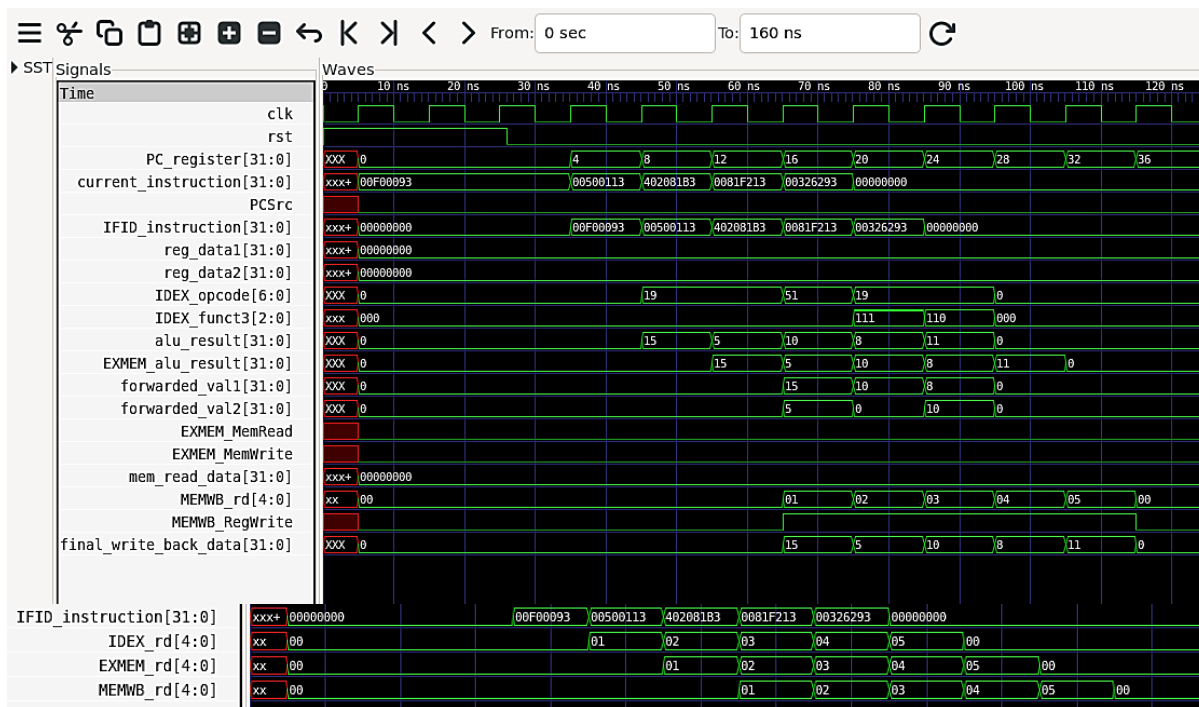
```

```

$display("Decode | Opcode: %b | rs1: %0d | rs2: %0d",
        uut.IDEX_opcode, uut.IDEX_rs1, uut.IDEX_rs2);
$display("Exec | ALU Out (Hex): %h | ALU Out (Dec): %0d",
        uut.EXMEM_alu_result, uut.EXMEM_alu_result);

// Printing in both Decimal (%0d) and Hexadecimal (%h) for clarity
$display("Status | Reg[x1] = %0d (Hex: %h)", uut.Register_File[1], uut.Register_File[1]);
$display("Status | Reg[x2] = %0d (Hex: %h)", uut.Register_File[2], uut.Register_File[2]);
$display("Status | Reg[x3] = %0d (Hex: %h) [TRAP]", uut.Register_File[3], uut.Register_File[3]);
$display("Status | Reg[x4] = %0d (Hex: %h) [TARGET]", uut.Register_File[4], uut.Register_File[4]);
$display("-----\n");
end
end
endmodule

```



```

--- Time: 30000 ns (Falling Clock Edge) ---
Fetch | PC: 0 | Raw Inst: 03700093
Decode | Opcode: 0000000 | rs1: 0 | rs2: 0
Exec | ALU Out (Hex): 00000000 | ALU Out (Dec): 0
Status | Reg[x1] = 0 (Hex: 00000000)
Status | Reg[x2] = 0 (Hex: 00000000)
Status | Reg[x3] = 0 (Hex: 00000000) [TRAP]
Status | Reg[x4] = 0 (Hex: 00000000) [TARGET]
-----

```

```

--- Time: 40000 ns (Falling Clock Edge) ---
Fetch | PC: 4 | Raw Inst: 03700113
Decode | Opcode: 0000000 | rs1: 0 | rs2: 0
Exec | ALU Out (Hex): 00000000 | ALU Out (Dec): 0
Status | Reg[x1] = 0 (Hex: 00000000)
Status | Reg[x2] = 0 (Hex: 00000000)
Status | Reg[x3] = 0 (Hex: 00000000) [TRAP]
Status | Reg[x4] = 0 (Hex: 00000000) [TARGET]
-----

```

```

--- Time: 50000 ns (Falling Clock Edge) ---
Fetch | PC: 8 | Raw Inst: 00208463
Decode | Opcode: 0010011 | rs1: 0 | rs2: 23
Exec | ALU Out (Hex): 00000000 | ALU Out (Dec): 0
Status | Reg[x1] = 0 (Hex: 00000000)
Status | Reg[x2] = 0 (Hex: 00000000)
Status | Reg[x3] = 0 (Hex: 00000000) [TRAP]
Status | Reg[x4] = 0 (Hex: 00000000) [TARGET]
-----

```

--- Time: 60000 ns (Falling Clock Edge) ---
Fetch | PC: 12 | Raw Inst: 06300193
Decode | Opcode: 0010011 | rs1: 0 | rs2: 23
Exec | ALU Out (Hex): 00000037 | ALU Out (Dec): 55
Status | Reg[x1] = 0 (Hex: 00000000)
Status | Reg[x2] = 0 (Hex: 00000000)
Status | Reg[x3] = 0 (Hex: 00000000) [TRAP]
Status | Reg[x4] = 0 (Hex: 00000000) [TARGET]

--- Time: 70000 ns (Falling Clock Edge) ---
Fetch | PC: 16 | Raw Inst: 01600213
Decode | Opcode: 1100011 | rs1: 1 | rs2: 2
Exec | ALU Out (Hex): 00000037 | ALU Out (Dec): 55
Status | Reg[x1] = 0 (Hex: 00000000)
Status | Reg[x2] = 0 (Hex: 00000000)
Status | Reg[x3] = 0 (Hex: 00000000) [TRAP]
Status | Reg[x4] = 0 (Hex: 00000000) [TARGET]

--- Time: 80000 ns (Falling Clock Edge) ---
Fetch | PC: 16 | Raw Inst: 01600213
Decode | Opcode: 0000000 | rs1: 0 | rs2: 0
Exec | ALU Out (Hex): 00000000 | ALU Out (Dec): 0
Status | Reg[x1] = 55 (Hex: 00000037)
Status | Reg[x2] = 0 (Hex: 00000000)
Status | Reg[x3] = 0 (Hex: 00000000) [TRAP]
Status | Reg[x4] = 0 (Hex: 00000000) [TARGET]

--- Time: 90000 ns (Falling Clock Edge) ---
Fetch | PC: 20 | Raw Inst: 00000000
Decode | Opcode: 0000000 | rs1: 0 | rs2: 0
Exec | ALU Out (Hex): 00000000 | ALU Out (Dec): 0
Status | Reg[x1] = 55 (Hex: 00000037)
Status | Reg[x2] = 55 (Hex: 00000037)
Status | Reg[x3] = 0 (Hex: 00000000) [TRAP]
Status | Reg[x4] = 0 (Hex: 00000000) [TARGET]

--- Time: 100000 ns (Falling Clock Edge) ---
Fetch | PC: 24 | Raw Inst: 00000000
Decode | Opcode: 0010011 | rs1: 0 | rs2: 22
Exec | ALU Out (Hex): 00000000 | ALU Out (Dec): 0
Status | Reg[x1] = 55 (Hex: 00000037)
Status | Reg[x2] = 55 (Hex: 00000037)
Status | Reg[x3] = 0 (Hex: 00000000) [TRAP]
Status | Reg[x4] = 0 (Hex: 00000000) [TARGET]

--- Time: 110000 ns (Falling Clock Edge) ---
Fetch | PC: 28 | Raw Inst: xxxxxxxx
Decode | Opcode: 0000000 | rs1: 0 | rs2: 0
Exec | ALU Out (Hex): 00000016 | ALU Out (Dec): 22
Status | Reg[x1] = 55 (Hex: 00000037)
Status | Reg[x2] = 55 (Hex: 00000037)
Status | Reg[x3] = 0 (Hex: 00000000) [TRAP]
Status | Reg[x4] = 0 (Hex: 00000000) [TARGET]

--- Time: 120000 ns (Falling Clock Edge) ---
Fetch | PC: 32 | Raw Inst: xxxxxxxx
Decode | Opcode: 0000000 | rs1: 0 | rs2: 0
Exec | ALU Out (Hex): 00000000 | ALU Out (Dec): 0
Status | Reg[x1] = 55 (Hex: 00000037)
Status | Reg[x2] = 55 (Hex: 00000037)
Status | Reg[x3] = 0 (Hex: 00000000) [TRAP]
Status | Reg[x4] = 0 (Hex: 00000000) [TARGET]

--- Time: 130000 ns (Falling Clock Edge) ---
Fetch | PC: 36 | Raw Inst: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1: x | rs2: x
Exec | ALU Out (Hex): 00000000 | ALU Out (Dec): 0
Status | Reg[x1] = 55 (Hex: 00000037)
Status | Reg[x2] = 55 (Hex: 00000037)
Status | Reg[x3] = 0 (Hex: 00000000) [TRAP]
Status | Reg[x4] = 22 (Hex: 00000016) [TARGET]

--- Time: 140000 ns (Falling Clock Edge) ---
Fetch | PC: x | Raw Inst: xxxxxxxx
Decode | Opcode: xxxxxxxx | rs1: x | rs2: x

```

Exec    | ALU Out (Hex): 00000000 | ALU Out (Dec): 0
Status  | Reg[x1] = 55 (Hex: 00000037)
Status  | Reg[x2] = 55 (Hex: 00000037)
Status  | Reg[x3] = 0 (Hex: 00000000) [TRAP]
Status  | Reg[x4] = 22 (Hex: 00000016) [TARGET]
-----

```

```

--- Time: 150000 ns (Falling Clock Edge) ---
Fetch   | PC: x | Raw Inst: xxxxxxxx
Decode  | Opcode: xxxxxxxx | rs1: x | rs2: x
Exec    | ALU Out (Hex): 00000000 | ALU Out (Dec): 0
Status  | Reg[x1] = 55 (Hex: 00000037)
Status  | Reg[x2] = 55 (Hex: 00000037)
Status  | Reg[x3] = 0 (Hex: 00000000) [TRAP]
Status  | Reg[x4] = 22 (Hex: 00000016) [TARGET]
-----

```

```

--- Time: 160000 ns (Falling Clock Edge) ---
Fetch   | PC: x | Raw Inst: xxxxxxxx
Decode  | Opcode: xxxxxxxx | rs1: x | rs2: x
Exec    | ALU Out (Hex): 00000000 | ALU Out (Dec): 0
Status  | Reg[x1] = 55 (Hex: 00000037)
Status  | Reg[x2] = 55 (Hex: 00000037)
Status  | Reg[x3] = 0 (Hex: 00000000) [TRAP]
Status  | Reg[x4] = 22 (Hex: 00000016) [TARGET]
-----

```

Final: A 5-stage Pipelined Processor

```

module pipelined_processor(
    input clk,
    input rst
);

// =====
// 1. SIGNAL & ARCHITECTURAL REGISTERS DECLARATIONS
// =====

wire [31:0] Nxt_PC;
reg [31:0] PC_register;

// Memory Sub-systems
reg [7:0] Inst_memory [0:255];
reg [7:0] Data_memory [0:255];

// Core Decoding & Operational Wires
wire [31:0] current_instruction;
reg MemRead, MemWrite;
reg RegWrite, MemtoReg;
reg [31:0] alu_result;
wire [31:0] mem_read_data;
wire [31:0] final_write_back_data;

// Control Hazard Resolution Line
wire PCSrc;

// Register File Storage & Ports
reg [31:0] Register_File [0:31];
wire [31:0] reg_data1;
wire [31:0] reg_data2;

// -----
// PIPELINE REGISTER FIELDS (Stage Boundary Latches)
// -----

```

```

// IF/ID Boundary Registers
reg [31:0] IFID_instruction;
reg [31:0] IFID_PC;

// ID/EX Boundary Registers
reg [31:0] IDEX_reg_data1;
reg [31:0] IDEX_reg_data2;
reg [31:0] IDEX_imm_ext;
reg [4:0] IDEX_rs1;
reg [4:0] IDEX_rs2;
reg [4:0] IDEX_rd;
reg [2:0] IDEX_funct3;
reg [6:0] IDEX_funct7;
reg [6:0] IDEX_opcode;
reg [31:0] IDEX_PC;
reg IDEX_MemRead;
reg IDEX_MemWrite;
reg IDEX_RegWrite;
reg IDEX_MemtoReg;

// EX/MEM Boundary Registers
reg [31:0] EXMEM_alu_result;
reg [31:0] EXMEM_reg_data2;
reg [4:0] EXMEM_rd;
reg [6:0] EXMEM_opcode;
reg EXMEM_MemRead;
reg EXMEM_MemWrite;
reg EXMEM_RegWrite;
reg EXMEM_MemtoReg;

// MEM/WB Boundary Registers
reg [31:0] MEMWB_alu_result;
reg [31:0] MEMWB_mem_read_data;
reg [4:0] MEMWB_rd;
reg [6:0] MEMWB_opcode;
reg MEMWB_RegWrite;
reg MEMWB_MemtoReg;

// Asynchronous Hardware Power-On Reset Initialization
integer i;
initial begin
    for (i = 0; i < 32; i = i + 1) begin
        Register_File[i] = 32'h0;
    end
end

// =====
// STAGE 1: INSTRUCTION FETCH (IF)
// =====

// Combinational Next PC Evaluation Loop (Chooses branch target if branch is taken)
assign Nxt_PC = (PCSrc) ? (IDEX_PC + IDEX_imm_ext) : (PC_register + 4);

// Synchronous PC Program Register State Machine
always @(posedge clk) begin
    if (rst)
        PC_register <= 32'h0;

```

```

else
    PC_register <= Nxt_PC;
end

// Combinational Little-Endian Byte Stitching from Program ROM
assign current_instruction = { Inst_memory[PC_register + 3],
                               Inst_memory[PC_register + 2],
                               Inst_memory[PC_register + 1],
                               Inst_memory[PC_register] };

// Synchronous Pipeline Intermediary: IF/ID (With Control Hazard Flushing)
always @(posedge clk) begin
    if (rst || PCSrc) begin
        IFID_instruction <= 32'h0;
        IFID_PC <= 32'h0;
    end else begin
        IFID_instruction <= current_instruction;
        IFID_PC <= PC_register;
    end
end

// =====
// STAGE 2: INSTRUCTION DECODE / OPERAND FETCH (ID)
// =====

// Combinational Field Extraction Maps
wire [6:0] opcode = IFID_instruction[6:0];
wire [4:0] rd = IFID_instruction[11:7];
wire [2:0] funct3 = IFID_instruction[14:12];
wire [4:0] rs1 = IFID_instruction[19:15];
wire [4:0] rs2 = IFID_instruction[24:20];
wire [6:0] funct7 = IFID_instruction[31:25];

// Asymmetric Immediate Generator Logic (Supports I-Type, S-Type, and B-Type Layouts)
reg [31:0] imm_ext;
always @(*) begin
    if (opcode == 7'b0100011) begin
        // S-type instruction layout (sw)
        imm_ext = { {20{IFID_instruction[31]}}, IFID_instruction[31:25], IFID_instruction[11:7] };
    end else if (opcode == 7'b1100011) begin
        // B-type instruction layout (beq)
        imm_ext = { {20{IFID_instruction[31]}}, IFID_instruction[7], IFID_instruction[30:25], IFID_instruction[11:8],
1'b0 };
    end else begin
        // Standard I-type instruction layout (addi, lw, andi, ori)
        imm_ext = { {20{IFID_instruction[31]}}, IFID_instruction[31:20] };
    end
end

// Register File Asynchronous Dual Read Interface
assign reg_data1 = (rs1 == 5'b0) ? 32'h0 : Register_File[rs1];
assign reg_data2 = (rs2 == 5'b0) ? 32'h0 : Register_File[rs2];

// Combinational Main Control Decoder
always @(*) begin
    MemRead = 1'b0;
    MemWrite = 1'b0;

```

```

RegWrite = 1'b0;
MemtoReg = 1'b0;

case (opcode)
  7'b0110011: begin // R-type (add, sub, and, or)
    RegWrite = 1'b1;
    MemtoReg = 1'b0;
  end
  7'b0010011: begin // I-type arithmetic & logic (addi, andi, ori)
    RegWrite = 1'b1;
    MemtoReg = 1'b0;
  end
  7'b0000011: begin // lw
    MemRead = 1'b1;
    RegWrite = 1'b1;
    MemtoReg = 1'b1;
  end
  7'b0100011: begin // sw
    MemWrite = 1'b1;
    RegWrite = 1'b0;
    MemtoReg = 1'b0;
  end
  7'b1100011: begin // beq
    MemWrite = 1'b0;
    RegWrite = 1'b0;
    MemtoReg = 1'b0;
  end
endcase
end

```

// Synchronous Pipeline Intermediary: ID/EX

```

always @(posedge clk) begin
  if (rst) begin
    IDEX_reg_data1 <= 32'h0;
    IDEX_reg_data2 <= 32'h0;
    IDEX_imm_ext  <= 32'h0;
    IDEX_rd      <= 5'h0;
    IDEX_funct3   <= 3'h0;
    IDEX_funct7   <= 7'h0;
    IDEX_opcode   <= 7'h0;
    IDEX_rs1      <= 5'h0;
    IDEX_rs2      <= 5'h0;
    IDEX_PC       <= 32'h0;
    IDEX_MemRead  <= 1'b0;
    IDEX_MemWrite <= 1'b0;
    IDEX_RegWrite <= 1'b0;
    IDEX_MemtoReg <= 1'b0;
  end else begin
    IDEX_reg_data1 <= reg_data1;
    IDEX_reg_data2 <= reg_data2;
    IDEX_imm_ext  <= imm_ext;
    IDEX_rd      <= rd;
    IDEX_funct3   <= funct3;
    IDEX_funct7   <= funct7;
    IDEX_rs1      <= rs1;
    IDEX_rs2      <= rs2;
    IDEX_PC       <= IFID_PC;
  end
end

```

```

    if (PCSrc) begin
        // Direct clean single-path NOP bubble override injection
        IDEX_opcode  <= 7'b0;
        IDEX_MemRead  <= 1'b0;
        IDEX_MemWrite <= 1'b0;
        IDEX_RegWrite <= 1'b0;
        IDEX_MemtoReg <= 1'b0;
    end else begin
        IDEX_opcode  <= opcode;
        IDEX_MemRead  <= MemRead;
        IDEX_MemWrite <= MemWrite;
        IDEX_RegWrite <= RegWrite;
        IDEX_MemtoReg <= MemtoReg;
    end
end
end

// =====
// STAGE 3: EXECUTE / ADDRESS GENERATION (EX)
// =====

reg [31:0] forwarded_val1;
reg [31:0] forwarded_val2;
wire [31:0] alu_operand2;

// Operand 1 Forwarding Matrix Selector Loop
always @(*) begin
    if (EXMEM_rd != 5'b0 && EXMEM_rd == IDEX_rs1 && EXMEM_RegWrite) begin
        forwarded_val1 = EXMEM_alu_result; // EX-to-EX Hazard
    end else if (MEMWB_rd != 5'b0 && MEMWB_rd == IDEX_rs1 && MEMWB_RegWrite) begin
        forwarded_val1 = final_write_back_data; // MEM-to-EX Hazard (Bridges WB to EX stage)
    end else begin
        forwarded_val1 = IDEX_reg_data1;
    end
end

// Operand 2 Forwarding Matrix Selector Loop
always @(*) begin
    if (EXMEM_rd != 5'b0 && EXMEM_rd == IDEX_rs2 && EXMEM_RegWrite) begin
        forwarded_val2 = EXMEM_alu_result; // FIXED: Successfully isolated from forwarded_val1
    end else if (MEMWB_rd != 5'b0 && MEMWB_rd == IDEX_rs2 && MEMWB_RegWrite) begin
        forwarded_val2 = final_write_back_data; // MEM-to-EX Hazard (Bridges WB to EX stage)
    end else begin
        forwarded_val2 = IDEX_reg_data2;
    end
end

// ALU Secondary Operand Source Ingestion Selector Multiplexer
assign alu_operand2 = (IDEX_opcode == 7'b0110011) ? forwarded_val2 : IDEX_imm_ext;

// Control Hazard Decision: Assert PCSrc if instruction is a branch AND values match
assign PCSrc = (IDEX_opcode == 7'b1100011) && (forwarded_val1 == forwarded_val2);

// Expanded Mathematical ALU Calculation Engine (Nested Case Matrix)
always @(*) begin
    alu_result = 32'h0;

```

```

case (IDEX_opcode)
  7'b0110011: begin // R-type Instructions (Register-to-Register)
    case (IDEX_funct3)
      3'b000: begin
        if (IDEX_funct7 == 7'b0100000)
          alu_result = forwarded_val1 - forwarded_val2; // sub
        else
          alu_result = forwarded_val1 + forwarded_val2; // add
        end
      3'b110: alu_result = forwarded_val1 | forwarded_val2; // or
      3'b111: alu_result = forwarded_val1 & forwarded_val2; // and
      default: alu_result = 32'h0;
    endcase
  end

  7'b0010011: begin // I-type Arithmetic & Logical Instructions (Register-Immediate)
    case (IDEX_funct3)
      3'b000: alu_result = forwarded_val1 + alu_operand2; // addi
      3'b110: alu_result = forwarded_val1 | alu_operand2; // ori
      3'b111: alu_result = forwarded_val1 & alu_operand2; // andi
      default: alu_result = 32'h0;
    endcase
  end

  7'b0000011, 7'b0100011: begin
    alu_result = forwarded_val1 + alu_operand2; // lw / sw Address Generation
  end

  default: alu_result = 32'h0;
endcase
end

// Synchronous Pipeline Intermediary: EX/MEM
always @(posedge clk) begin
  if (rst) begin
    EXMEM_alu_result <= 32'h0;
    EXMEM_reg_data2 <= 32'h0;
    EXMEM_rd <= 5'h0;
    EXMEM_opcode <= 7'h0;
    EXMEM_MemRead <= 1'b0;
    EXMEM_MemWrite <= 1'b0;
    EXMEM_RegWrite <= 1'b0;
    EXMEM_MemtoReg <= 1'b0;
  end else begin
    EXMEM_alu_result <= alu_result;
    EXMEM_reg_data2 <= forwarded_val2;
    EXMEM_rd <= IDEX_rd;

    if (PCSrc) begin
      // Wipes out trailing hazard control instructions smoothly
      EXMEM_opcode <= 7'b0;
      EXMEM_MemRead <= 1'b0;
      EXMEM_MemWrite <= 1'b0;
      EXMEM_RegWrite <= 1'b0;
      EXMEM_MemtoReg <= 1'b0;
    end else begin
      EXMEM_opcode <= IDEX_opcode;
    end
  end
end

```

```

    EXMEM_MemRead  <= IDEX_MemRead;
    EXMEM_MemWrite <= IDEX_MemWrite;
    EXMEM_RegWrite <= IDEX_RegWrite;
    EXMEM_MemtoReg <= IDEX_MemtoReg;
end
end
end

// =====
// STAGE 4: DATA MEMORY ACCESS (MEM)
// =====
assign mem_read_data = (EXMEM_MemRead) ? { Data_memory[EXMEM_alu_result + 3],
                                           Data_memory[EXMEM_alu_result + 2],
                                           Data_memory[EXMEM_alu_result + 1],
                                           Data_memory[EXMEM_alu_result] } : 32'h0;

always @(posedge clk) begin
    if (EXMEM_MemWrite) begin
        Data_memory[EXMEM_alu_result]  <= EXMEM_reg_data2[7:0];
        Data_memory[EXMEM_alu_result + 1] <= EXMEM_reg_data2[15:8];
        Data_memory[EXMEM_alu_result + 2] <= EXMEM_reg_data2[23:16];
        Data_memory[EXMEM_alu_result + 3] <= EXMEM_reg_data2[31:24];
    end
end

always @(posedge clk) begin
    if (rst) begin
        MEMWB_alu_result  <= 32'h0;
        MEMWB_mem_read_data <= 32'h0;
        MEMWB_rd          <= 5'h0;
        MEMWB_opcode      <= 7'h0;
        MEMWB_RegWrite    <= 1'b0;
        MEMWB_MemtoReg    <= 1'b0;
    end else begin
        MEMWB_alu_result  <= EXMEM_alu_result;
        MEMWB_mem_read_data <= mem_read_data;
        MEMWB_rd          <= EXMEM_rd;
        MEMWB_opcode      <= EXMEM_opcode;
        MEMWB_RegWrite    <= EXMEM_RegWrite;
        MEMWB_MemtoReg    <= EXMEM_MemtoReg;
    end
end

// =====
// STAGE 5: WRITE-BACK (WB)
// =====
assign final_write_back_data = (MEMWB_MemtoReg) ? MEMWB_mem_read_data : MEMWB_alu_result;

always @(posedge clk) begin
    if (!rst && MEMWB_RegWrite && MEMWB_rd != 5'h0) begin
        Register_File[MEMWB_rd] <= final_write_back_data;
    end
end
end

```

Testbench: `timescale 1ns / 1ps

module tb_pipelined_processor();

```

reg clk;
reg rst;

pipelined_processor uut (
    .clk(clk),
    .rst(rst)
);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_pipelined_processor);
end

always begin
    #5 clk = ~clk;
end

// Dynamic File Ingestion Routine
initial begin
    $readmemh("program.hex", uut.Inst_memory);
end

initial begin
    clk = 0;
    rst = 1;
    #20;
    @(posedge clk);
    #1 rst = 0;

    repeat (30) @(posedge clk);
    $finish;
end

// =====
// CORRECTED FALLING EDGE MONITOR: Captures settled, accurate states
// =====
always @(negedge clk) begin
    if (!rst) begin
        $display("--- Time: %0t ns (Setted State) ---", $time);

        // 1. Fetch Stage: Look at the actual un-latched wire values
        $display("Fetch | PC: %0d | Raw Inst: %h", uut.PC_register, uut.current_instruction);

        // 2. Decode Stage: Look at what is CURRENTLY inside the IFID register being decoded
        $display("Decode | Opcode: %b | rs1: %0d | rs2: %0d | rd: %0d", uut.opcode, uut.rs1, uut.rs2, uut.rd);

        // 3. Execute Stage: Look at the live ALU output calculation right now
        $display("Exec | ALU Result/Target: %0d", uut.alu_result);

        // 4. Memory Stage: Look at the active RAM pins right now
        $display("Memory | MemRead En: %b | MemWrite En: %b | Raw RAM Read Out: %0d",
uut.EXMEM_MemRead, uut.EXMEM_MemWrite, uut.mem_read_data);

        // 5. Architectural State Snapshots
        $display("Status | Register File Snapshot:");
    end
end

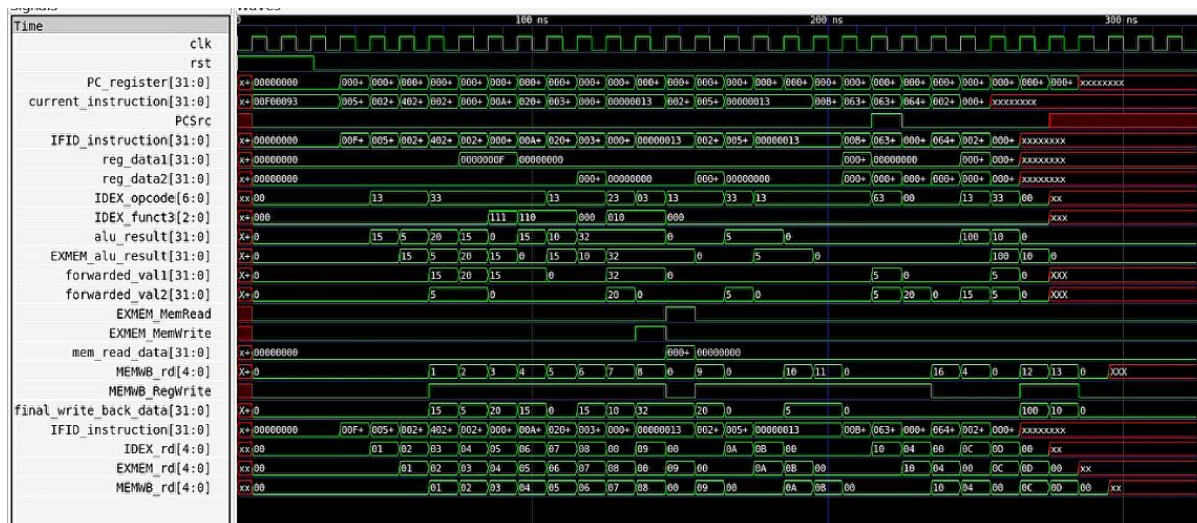
```

```

$display("    | x1=%0d | x2=%0d | x3=%0d | x4=%0d | x5=%0d", uut.Register_File[1], uut.Register_File[2],
uut.Register_File[3], uut.Register_File[4], uut.Register_File[5]);
$display("    | x6=%0d | x7=%0d | x8=%0d | x9=%0d | x10=%0d", uut.Register_File[6], uut.Register_File[7],
uut.Register_File[8], uut.Register_File[9], uut.Register_File[10]);
$display("    | x11=%0d | x12=%0d | x13=%0d | x14=%0d | x15=%0d", uut.Register_File[11],
uut.Register_File[12], uut.Register_File[13], uut.Register_File[14], uut.Register_File[15]);

$display("Status | Target Data Memory Segments:");
$display("    | RAM[0]=%0d | RAM[4]=%0d | RAM[32]=%0d", uut.Data_memory[0], uut.Data_memory[4],
uut.Data_memory[32]);
$display("-----\n");
end
end
endmodule
endmodule

```



WARNING: tb_pipelined_processor.v:24: \$readmemh(program.hex): Not enough words in the file for the requested range [0:255].

--- Time: 30000 ns (Setted State) ---

Fetch | PC: 0 | Raw Inst: 00f00093

Decode | Opcode: 00000000 | rs1: 0 | rs2: 0 | rd: 0

Exec | ALU Result/Target: 0

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0

Status | Register File Snapshot:

| x1=0 | x2=0 | x3=0 | x4=0 | x5=0

| x6=0 | x7=0 | x8=0 | x9=0 | x10=0

| x11=0 | x12=0 | x13=0 | x14=0 | x15=0

Status | Target Data Memory Segments:

| RAM[0]=x | RAM[4]=x | RAM[32]=x

--- Time: 40000 ns (Setted State) ---

Fetch | PC: 4 | Raw Inst: 00500113

Decode | Opcode: 0010011 | rs1: 0 | rs2: 15 | rd: 1

Exec | ALU Result/Target: 0

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0

Status | Register File Snapshot:

| x1=0 | x2=0 | x3=0 | x4=0 | x5=0

| x6=0 | x7=0 | x8=0 | x9=0 | x10=0

| x11=0 | x12=0 | x13=0 | x14=0 | x15=0

Status | Target Data Memory Segments:

| RAM[0]=x | RAM[4]=x | RAM[32]=x

--- Time: 50000 ns (Setted State) ---

Fetch | PC: 8 | Raw Inst: 002081b3

Decode | Opcode: 0010011 | rs1: 0 | rs2: 5 | rd: 2

Exec | ALU Result/Target: 15

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0

Status | Register File Snapshot:

| x1=0 | x2=0 | x3=0 | x4=0 | x5=0

| x6=0 | x7=0 | x8=0 | x9=0 | x10=0

| x11=0 | x12=0 | x13=0 | x14=0 | x15=0

Status | Target Data Memory Segments:

| RAM[0]=x | RAM[4]=x | RAM[32]=x

--- Time: 60000 ns (Setted State) ---

Fetch | PC: 12 | Raw Inst: 40218233

Decode | Opcode: 0110011 | rs1: 1 | rs2: 2 | rd: 3

Exec | ALU Result/Target: 5

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0

Status | Register File Snapshot:

| x1=0 | x2=0 | x3=0 | x4=0 | x5=0

| x6=0 | x7=0 | x8=0 | x9=0 | x10=0

| x11=0 | x12=0 | x13=0 | x14=0 | x15=0

Status | Target Data Memory Segments:

| RAM[0]=x | RAM[4]=x | RAM[32]=x

--- Time: 70000 ns (Setted State) ---

Fetch | PC: 16 | Raw Inst: 0020f2b3

Decode | Opcode: 0110011 | rs1: 3 | rs2: 2 | rd: 4

Exec | ALU Result/Target: 20

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0

Status | Register File Snapshot:

| x1=0 | x2=0 | x3=0 | x4=0 | x5=0

| x6=0 | x7=0 | x8=0 | x9=0 | x10=0

| x11=0 | x12=0 | x13=0 | x14=0 | x15=0

Status | Target Data Memory Segments:

| RAM[0]=x | RAM[4]=x | RAM[32]=x

--- Time: 80000 ns (Setted State) ---

Fetch | PC: 20 | Raw Inst: 0000e333

Decode | Opcode: 0110011 | rs1: 1 | rs2: 2 | rd: 5

Exec | ALU Result/Target: 15

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0

Status | Register File Snapshot:

| x1=15 | x2=0 | x3=0 | x4=0 | x5=0

| x6=0 | x7=0 | x8=0 | x9=0 | x10=0

| x11=0 | x12=0 | x13=0 | x14=0 | x15=0

Status | Target Data Memory Segments:

| RAM[0]=x | RAM[4]=x | RAM[32]=x

--- Time: 90000 ns (Setted State) ---

Fetch | PC: 24 | Raw Inst: 00a06393

Decode | Opcode: 0110011 | rs1: 1 | rs2: 0 | rd: 6
Exec | ALU Result/Target: 0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=0 | x4=0 | x5=0
| x6=0 | x7=0 | x8=0 | x9=0 | x10=0
| x11=0 | x12=0 | x13=0 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=x

--- Time: 100000 ns (Setted State) ---
Fetch | PC: 28 | Raw Inst: 02000413
Decode | Opcode: 0010011 | rs1: 0 | rs2: 10 | rd: 7
Exec | ALU Result/Target: 15
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=0 | x5=0
| x6=0 | x7=0 | x8=0 | x9=0 | x10=0
| x11=0 | x12=0 | x13=0 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=x

--- Time: 110000 ns (Setted State) ---
Fetch | PC: 32 | Raw Inst: 00342023
Decode | Opcode: 0010011 | rs1: 0 | rs2: 0 | rd: 8
Exec | ALU Result/Target: 10
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=15 | x5=0
| x6=0 | x7=0 | x8=0 | x9=0 | x10=0
| x11=0 | x12=0 | x13=0 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=x

--- Time: 120000 ns (Setted State) ---
Fetch | PC: 36 | Raw Inst: 00042483
Decode | Opcode: 0100011 | rs1: 8 | rs2: 3 | rd: 0
Exec | ALU Result/Target: 32
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=15 | x5=0
| x6=0 | x7=0 | x8=0 | x9=0 | x10=0
| x11=0 | x12=0 | x13=0 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=x

--- Time: 130000 ns (Setted State) ---
Fetch | PC: 40 | Raw Inst: 00000013
Decode | Opcode: 0000011 | rs1: 8 | rs2: 0 | rd: 9
Exec | ALU Result/Target: 32
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=15 | x5=0

| x6=15 | x7=0 | x8=0 | x9=0 | x10=0
| x11=0 | x12=0 | x13=0 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=x

--- Time: 140000 ns (Setted State) ---
Fetch | PC: 44 | Raw Inst: 00000013
Decode | Opcode: 0010011 | rs1: 0 | rs2: 0 | rd: 0
Exec | ALU Result/Target: 32
Memory | MemRead En: 0 | MemWrite En: 1 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=15 | x5=0
| x6=15 | x7=10 | x8=0 | x9=0 | x10=0
| x11=0 | x12=0 | x13=0 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=x

--- Time: 150000 ns (Setted State) ---
Fetch | PC: 48 | Raw Inst: 00248533
Decode | Opcode: 0010011 | rs1: 0 | rs2: 0 | rd: 0
Exec | ALU Result/Target: 0
Memory | MemRead En: 1 | MemWrite En: 0 | Raw RAM Read Out: 20
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=15 | x5=0
| x6=15 | x7=10 | x8=32 | x9=0 | x10=0
| x11=0 | x12=0 | x13=0 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 160000 ns (Setted State) ---
Fetch | PC: 52 | Raw Inst: 00500593
Decode | Opcode: 0110011 | rs1: 9 | rs2: 2 | rd: 10
Exec | ALU Result/Target: 0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=15 | x5=0
| x6=15 | x7=10 | x8=32 | x9=0 | x10=0
| x11=0 | x12=0 | x13=0 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 170000 ns (Setted State) ---
Fetch | PC: 56 | Raw Inst: 00000013
Decode | Opcode: 0010011 | rs1: 0 | rs2: 5 | rd: 11
Exec | ALU Result/Target: 5
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=15 | x5=0
| x6=15 | x7=10 | x8=32 | x9=20 | x10=0
| x11=0 | x12=0 | x13=0 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 180000 ns (Setted State) ---

Fetch | PC: 60 | Raw Inst: 00000013

Decode | Opcode: 0010011 | rs1: 0 | rs2: 0 | rd: 0

Exec | ALU Result/Target: 5

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0

Status | Register File Snapshot:

| x1=15 | x2=5 | x3=20 | x4=15 | x5=0

| x6=15 | x7=10 | x8=32 | x9=20 | x10=0

| x11=0 | x12=0 | x13=0 | x14=0 | x15=0

Status | Target Data Memory Segments:

| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 190000 ns (Setted State) ---

Fetch | PC: 64 | Raw Inst: 00000013

Decode | Opcode: 0010011 | rs1: 0 | rs2: 0 | rd: 0

Exec | ALU Result/Target: 0

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0

Status | Register File Snapshot:

| x1=15 | x2=5 | x3=20 | x4=15 | x5=0

| x6=15 | x7=10 | x8=32 | x9=20 | x10=0

| x11=0 | x12=0 | x13=0 | x14=0 | x15=0

Status | Target Data Memory Segments:

| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 200000 ns (Setted State) ---

Fetch | PC: 68 | Raw Inst: 00b50863

Decode | Opcode: 0010011 | rs1: 0 | rs2: 0 | rd: 0

Exec | ALU Result/Target: 0

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0

Status | Register File Snapshot:

| x1=15 | x2=5 | x3=20 | x4=15 | x5=0

| x6=15 | x7=10 | x8=32 | x9=20 | x10=5

| x11=0 | x12=0 | x13=0 | x14=0 | x15=0

Status | Target Data Memory Segments:

| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 210000 ns (Setted State) ---

Fetch | PC: 72 | Raw Inst: 06300213

Decode | Opcode: 1100011 | rs1: 10 | rs2: 11 | rd: 16

Exec | ALU Result/Target: 0

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0

Status | Register File Snapshot:

| x1=15 | x2=5 | x3=20 | x4=15 | x5=0

| x6=15 | x7=10 | x8=32 | x9=20 | x10=5

| x11=5 | x12=0 | x13=0 | x14=0 | x15=0

Status | Target Data Memory Segments:

| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 220000 ns (Setted State) ---

Fetch | PC: 76 | Raw Inst: 06300293

Decode | Opcode: 0010011 | rs1: 0 | rs2: 3 | rd: 4

Exec | ALU Result/Target: 0

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=15 | x5=0
| x6=15 | x7=10 | x8=32 | x9=20 | x10=5
| x11=5 | x12=0 | x13=0 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 230000 ns (Setted State) ---
Fetch | PC: 84 | Raw Inst: 06400613
Decode | Opcode: 0000000 | rs1: 0 | rs2: 0 | rd: 0
Exec | ALU Result/Target: 0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=15 | x5=0
| x6=15 | x7=10 | x8=32 | x9=20 | x10=5
| x11=5 | x12=0 | x13=0 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 240000 ns (Setted State) ---
Fetch | PC: 88 | Raw Inst: 002506b3
Decode | Opcode: 0010011 | rs1: 0 | rs2: 4 | rd: 12
Exec | ALU Result/Target: 0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=15 | x5=0
| x6=15 | x7=10 | x8=32 | x9=20 | x10=5
| x11=5 | x12=0 | x13=0 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 250000 ns (Setted State) ---
Fetch | PC: 92 | Raw Inst: 00000000
Decode | Opcode: 0110011 | rs1: 10 | rs2: 2 | rd: 13
Exec | ALU Result/Target: 100
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=15 | x5=0
| x6=15 | x7=10 | x8=32 | x9=20 | x10=5
| x11=5 | x12=0 | x13=0 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 260000 ns (Setted State) ---
Fetch | PC: 96 | Raw Inst: xxxxxxxx
Decode | Opcode: 0000000 | rs1: 0 | rs2: 0 | rd: 0
Exec | ALU Result/Target: 10
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=15 | x5=0
| x6=15 | x7=10 | x8=32 | x9=20 | x10=5
| x11=5 | x12=0 | x13=0 | x14=0 | x15=0

Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 270000 ns (Setted State) ---

Fetch | PC: 100 | Raw Inst: xxxxxxxx

Decode | Opcode: xxxxxxxx | rs1: x | rs2: x | rd: x

Exec | ALU Result/Target: 0

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0

Status | Register File Snapshot:

| x1=15 | x2=5 | x3=20 | x4=15 | x5=0

| x6=15 | x7=10 | x8=32 | x9=20 | x10=5

| x11=5 | x12=0 | x13=0 | x14=0 | x15=0

Status | Target Data Memory Segments:

| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 280000 ns (Setted State) ---

Fetch | PC: 104 | Raw Inst: xxxxxxxx

Decode | Opcode: xxxxxxxx | rs1: x | rs2: x | rd: x

Exec | ALU Result/Target: 0

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0

Status | Register File Snapshot:

| x1=15 | x2=5 | x3=20 | x4=15 | x5=0

| x6=15 | x7=10 | x8=32 | x9=20 | x10=5

| x11=5 | x12=100 | x13=0 | x14=0 | x15=0

Status | Target Data Memory Segments:

| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 290000 ns (Setted State) ---

Fetch | PC: x | Raw Inst: xxxxxxxx

Decode | Opcode: xxxxxxxx | rs1: x | rs2: x | rd: x

Exec | ALU Result/Target: 0

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0

Status | Register File Snapshot:

| x1=15 | x2=5 | x3=20 | x4=15 | x5=0

| x6=15 | x7=10 | x8=32 | x9=20 | x10=5

| x11=5 | x12=100 | x13=10 | x14=0 | x15=0

Status | Target Data Memory Segments:

| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 300000 ns (Setted State) ---

Fetch | PC: x | Raw Inst: xxxxxxxx

Decode | Opcode: xxxxxxxx | rs1: x | rs2: x | rd: x

Exec | ALU Result/Target: 0

Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0

Status | Register File Snapshot:

| x1=15 | x2=5 | x3=20 | x4=15 | x5=0

| x6=15 | x7=10 | x8=32 | x9=20 | x10=5

| x11=5 | x12=100 | x13=10 | x14=0 | x15=0

Status | Target Data Memory Segments:

| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 310000 ns (Setted State) ---

Fetch | PC: x | Raw Inst: xxxxxxxx
Decode | Opcode: xxxxxxx | rs1: x | rs2: x | rd: x
Exec | ALU Result/Target: 0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=15 | x5=0
| x6=15 | x7=10 | x8=32 | x9=20 | x10=5
| x11=5 | x12=100 | x13=10 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=20

--- Time: 320000 ns (Setted State) ---

Fetch | PC: x | Raw Inst: xxxxxxxx
Decode | Opcode: xxxxxxx | rs1: x | rs2: x | rd: x
Exec | ALU Result/Target: 0
Memory | MemRead En: 0 | MemWrite En: 0 | Raw RAM Read Out: 0
Status | Register File Snapshot:
| x1=15 | x2=5 | x3=20 | x4=15 | x5=0
| x6=15 | x7=10 | x8=32 | x9=20 | x10=5
| x11=5 | x12=100 | x13=10 | x14=0 | x15=0
Status | Target Data Memory Segments:
| RAM[0]=x | RAM[4]=x | RAM[32]=20

tb_pipelined_processor.v:35: \$finish called at 325000 (1ps)